PRAKTIKUM 8
Pengembangan Aplikasi Client-Server berbasis Mobile Menggunakan Flutter

NAMA: Ihsanta Zaki Sanjaya

KELAS: 2 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Maretha Ruswiansari S.ST., M.T

MATA KULIAH: Praktikum Bahasa Pemrograman

TANGGAL: Kamis, 17 Oktober 2024

D4 TEKNIK KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2024

PRAKTIKUM 8

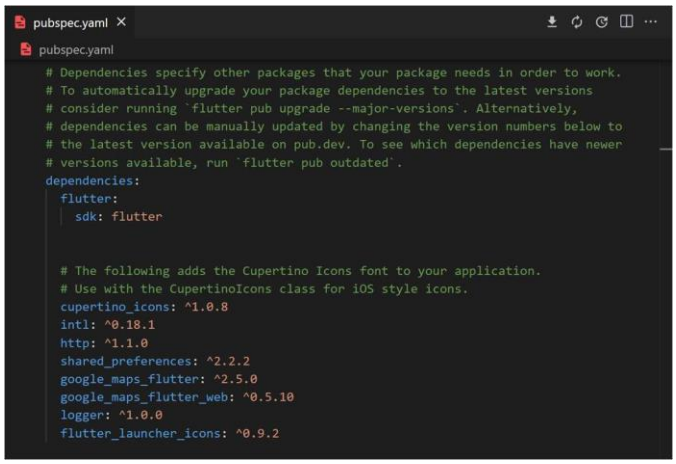
Pengembangan Aplikasi Client-Server berbasis Mobile Menggunakan Flutter

A. TUJUAN

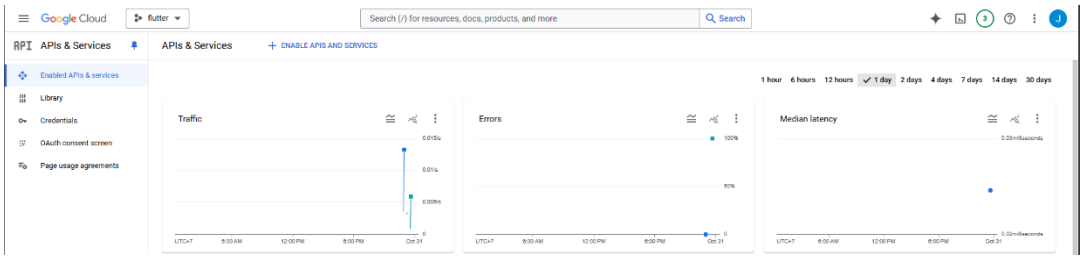
1. Mahasiswa dapat menjelaskan penerapan basis data dalam aplikasi.
2. Mahasiswa dapat mengembangkan *backend* dan *frontend* aplikasi.
3. Mahasiswa dapat menganalisis cara kerja aplikasi.

B. PROSEDUR

1. Buka VS Code, pilih **View > Command Palette** seperti berikut.
2. Beri nama *project*, misal **good_health**, kemudian tunggu hingga project selesai disiapkan.
3. Cari pada *project folder* 'good_health' untuk file **pubspec.yaml**, dimana file ini berisi metadata tentang package yang akan digunakan pada *project*. Tambahkan beberapa baris setelah 'cupertino.icons:', kemudian lanjutkan dengan *save file* dan tunggu proses sinkronisasi selesai.

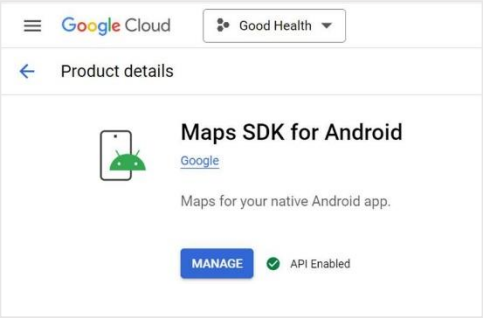


4. Pada pengembangan aplikasi ini membutuhkan Google Maps API Setup, sehingga perlu melakukan *setup* API key agar bisa menampilkan *maps* dalam *widget*:
   a. Android
      - Buka browser dan menuju ke halaman https://console.cloud.google.com/.
      - Pilih APIs & Services > Enabled APIs & Services.



      - Selanjutnya pilih CREATE PROJECT, isikan Good Health sebagai nama *project*.

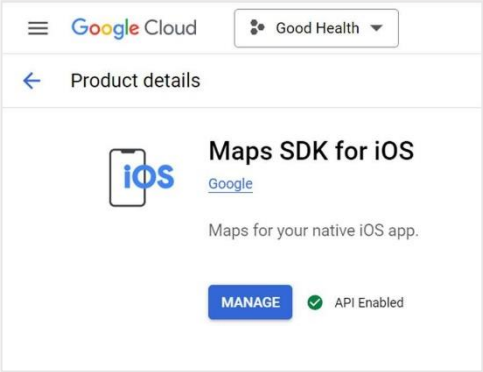
- Kemudian Pilih Maps SDK for Android dan klik *enable*.



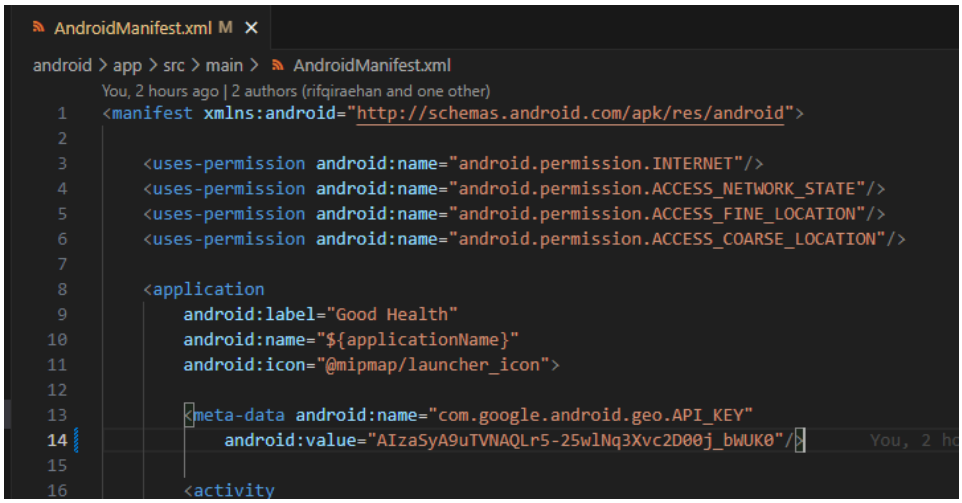
- Pilih Keys & Credentials > CREATE CREDENTIALS > API Key, kemudian copy API Key tersebut.

b. iOS

- Untuk iOS langkahnya sama, namun perlu mengaktifkan Maps SDK for iOS.



5. Selanjutnya buka *project* good_health > android > app > src > main > AndroidManifest.xml, kemudian tambahkan beberapa baris berikut:



6. Sedangkan untuk iOS dapat membuka *project* good_health > ios > Runner > Base.lproj > AppDelegate.Swift, kemudian tambahkan:

```
AppDelegate.swift M ×
ios > Runner > AppDelegate.swift
You, 1 second ago | 2 authors (rifqiraehan and one other)
1   import Flutter
2   import UIKit
3   import GoogleMaps
4
5   @main
6   @objc class AppDelegate: FlutterAppDelegate {
7     override func application(
8       _ application: UIApplication,
9       didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
10    ) -> Bool {
11      GMSServices.provideAPIKey("AIzaSyA9uTVNAQLr5-25wlNq3Xvc2D00j_bWUK0")        You, 1 se
12      GeneratedPluginRegistrant.register(with: self)
13      return super.application(application, didFinishLaunchingWithOptions: launchOptions)
14    }
15  }
```

7. Selanjutkan terkait *project organizer*, semua *source code* akan dibuat di dalam folder **lib**, seperti berikut:

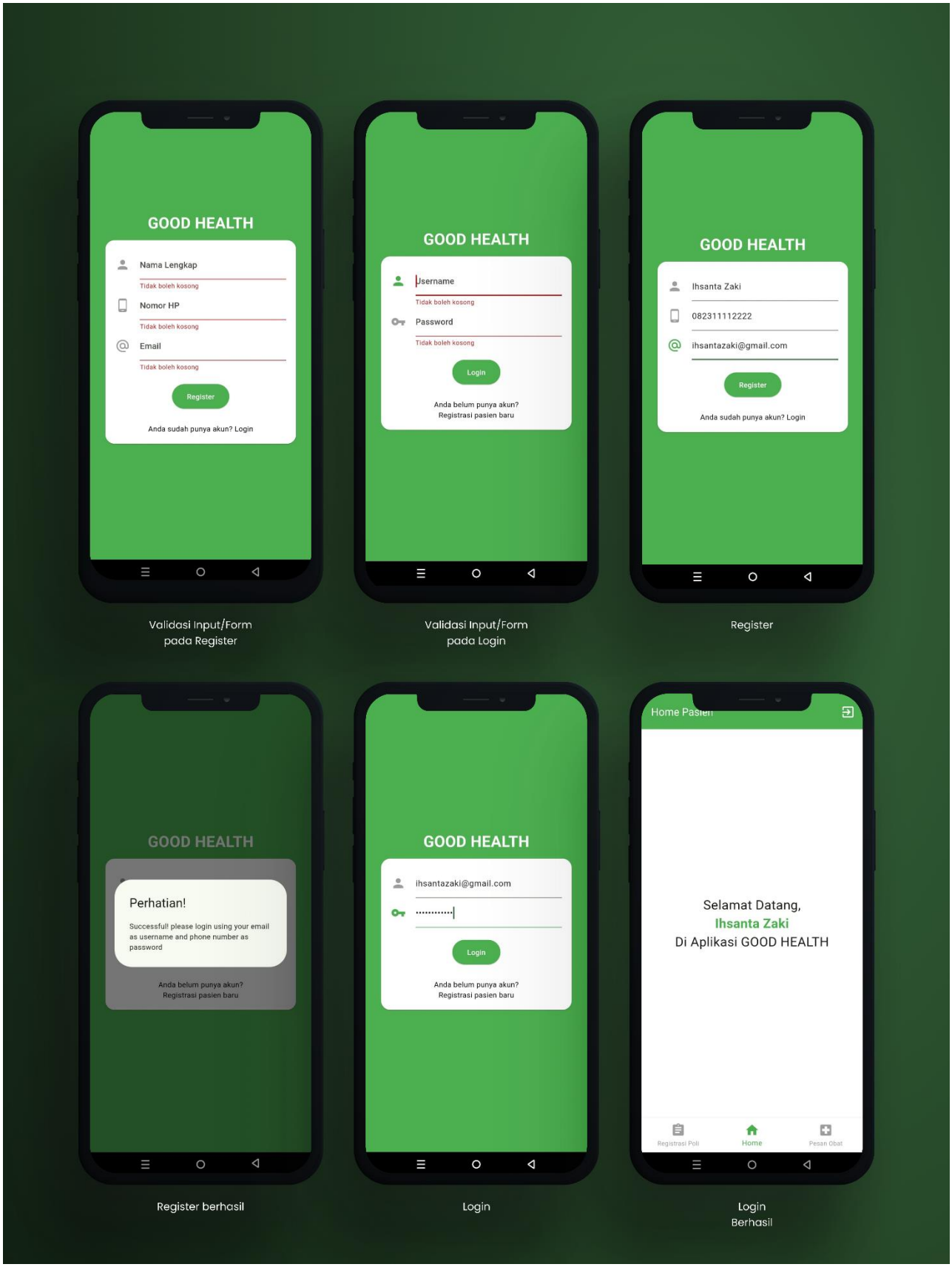
```
GOOD_HEALTH
  lib
    model
      dokter.dart
      obat.dart
      pasien.dart
      pesan_obat.dart
      reg_poli.dart
      user.dart
    page
      list_widget
        pesan_obat_pegawai.dart
        pesan_obat.dart
        reg_poli.dart
      modul_pasien
        home
          home_content.dart
        pesan_obat
          create.dart
          pesan_obat_content.dart
        reg_poli
          create.dart
          reg_poli_content.dart
        index.dart
        signup.dart
      modul_pegawai
        index.dart
        pesan_obat_view.dart
      login.dart
    util
      config.dart
      session.dart
      util.dart
    main.dart
```

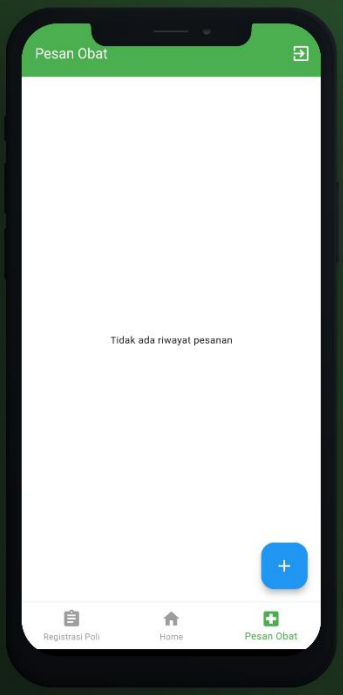
8. Selanjutnya buat folder baru dengan nama **model**, dimana dalam folder baru ini perlu membuat 6 file baru yaitu: **dokter.dart**, **obat.dart**, **pasien.dart**, **pesan_obat.dart**, **reg_poli.dart**, **user.dart**.

9. Untuk *session and utility*, maka perlu membuat folder **util**, dimana perlu menambah 2 file baru **session.dart** dan **util.dart**.
10. Berikutnya kita akan membuat file **main.dart** dan edit file.
11. Untuk membuat *login interface*, buat folder baru **page**, dan buat file **login.dart**.
12. Untuk membuat *register interface*, kita akan membuat membuat folder **modul_pasien** dan buat file baru **signup.dart**.
13. Nantinya user pasien akan melihat 3 menu yaitu Home, Registrasi Poli, dan Pesan Obat, maka kita buat 3 folder baru yaitu : **reg_poli**, **home**, dan **modul_pasien**, kemudian di dalam folder **modul_pasien**, buatlah file **index.dart**.
14. User tidak hanya pasien tetapi juga pegawai, maka kita buat folder baru bernama **modul_pegawai**, yang didalamnya akan terdapat file **index.dart**.
15. Selanjutnya, kita perlu membuat folder baru dengan nama **list_widget** di dalam folder **page**, kemudian buat file baru **pesan_obat_pegawai.dart**, **pesan_obat.dart**, dan **reg_poli.dart**.
16. Dalam folder **modul_pegawai**, buat file baru dengan nama **pesan_obat_view.dart**.
17. Dalam folder **modul_pasien**, kita buat folder baru **reg_poli** tambahkan file baru **reg_poli_content.dart**.
18. Untuk widget, kita perlu membuat file baru **reg_poli.dart** dalam folder **list_widget**.
19. Selanjutnya tambahkan file dengan nama **create.dart** dalam folder **reg_poli**.
20. Buat file baru dengan nama **pesan_obat_content.dart** dalam folder **pesan_obat**.
21. Buat file baru dengan nama **pesan_obat.dart** di dalam folder **list_widget**.
22. Tambahkan file dengan nama **create.dart** dalam folder **pesan_obat**.

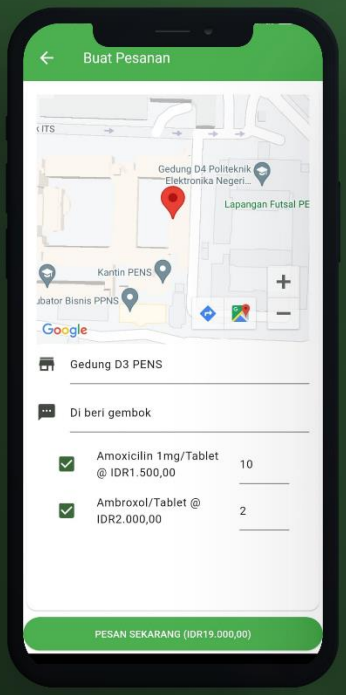
C. HASIL PERCOBAAN

a. Tampilan pada platform Android



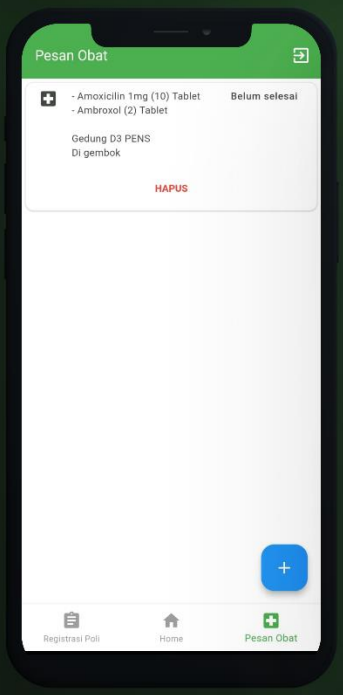


Page Pesan Obat
Default

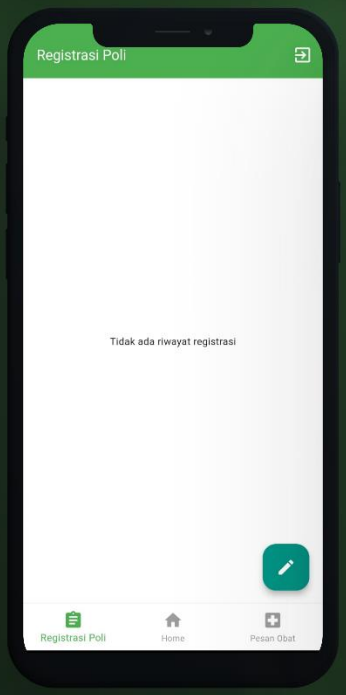


Menambah pesanan
Obat

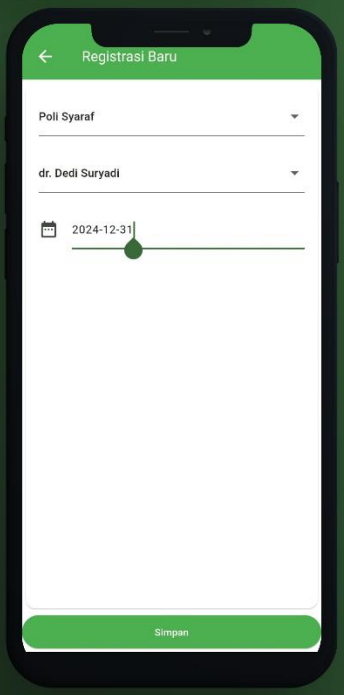
Notifikasi penambahan
pemesanan berhasil



Page Pesan Obat
Ada Pesanan

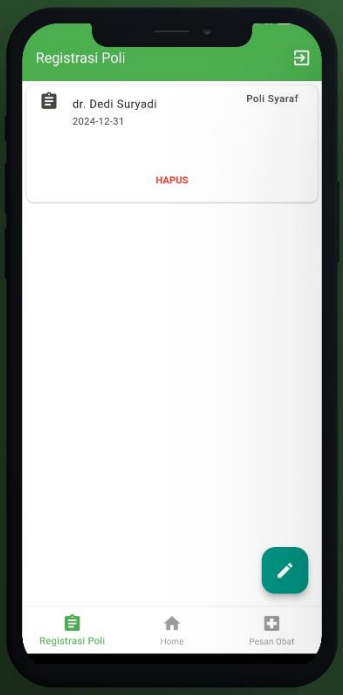


Page Registrasi Poli
Default

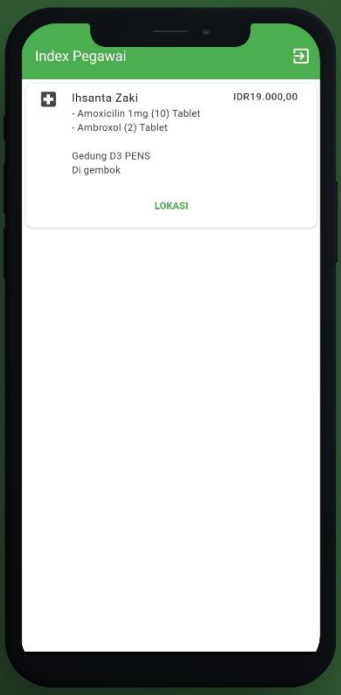


Menambah registrasi poli

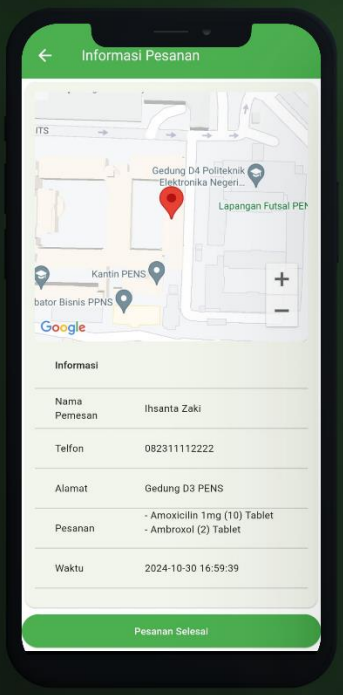
Notifikasi registrasi
poli berhasil



Registrasi Poli
Telah tercatat

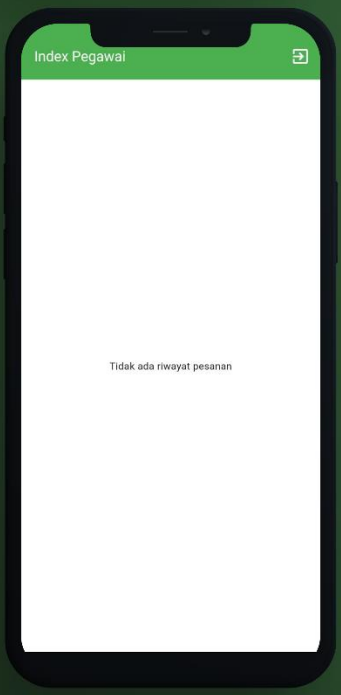


Login dengan
user Pegawai



Informasi pesanan
dari user Pegawai

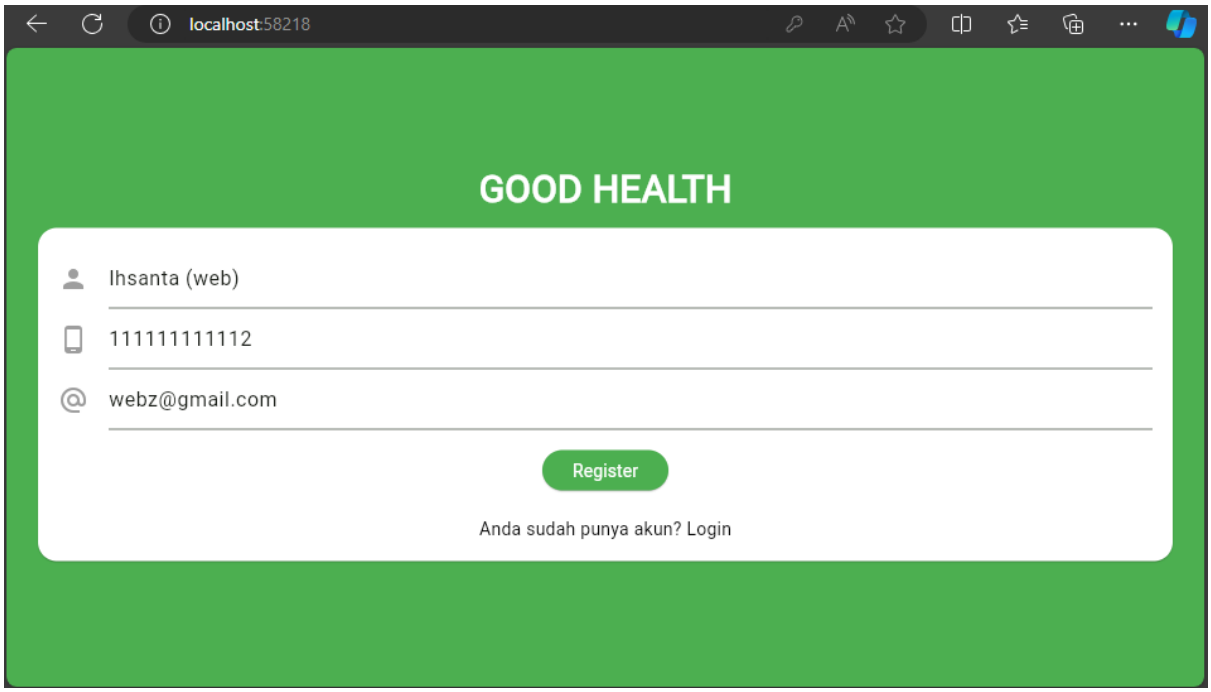
Notifikasi pesanan
berhasil diselesaikan



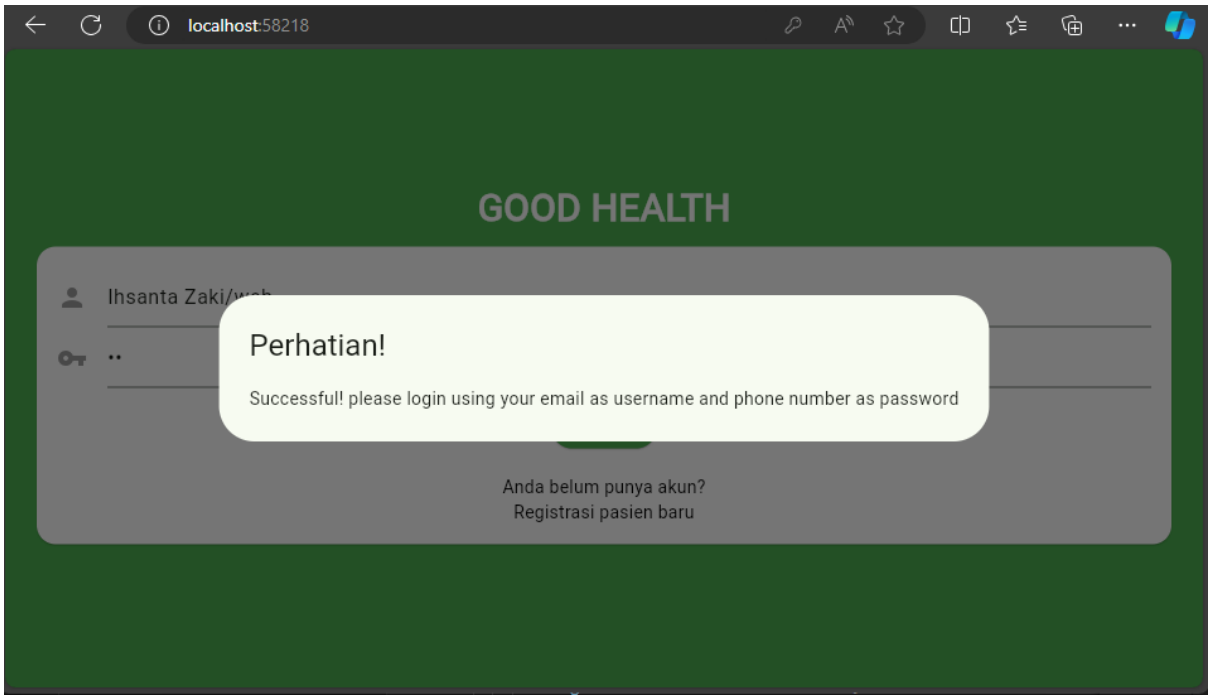
Pesanan kosong/habis

b. Tampilan pada platform Web Browser (Edge)

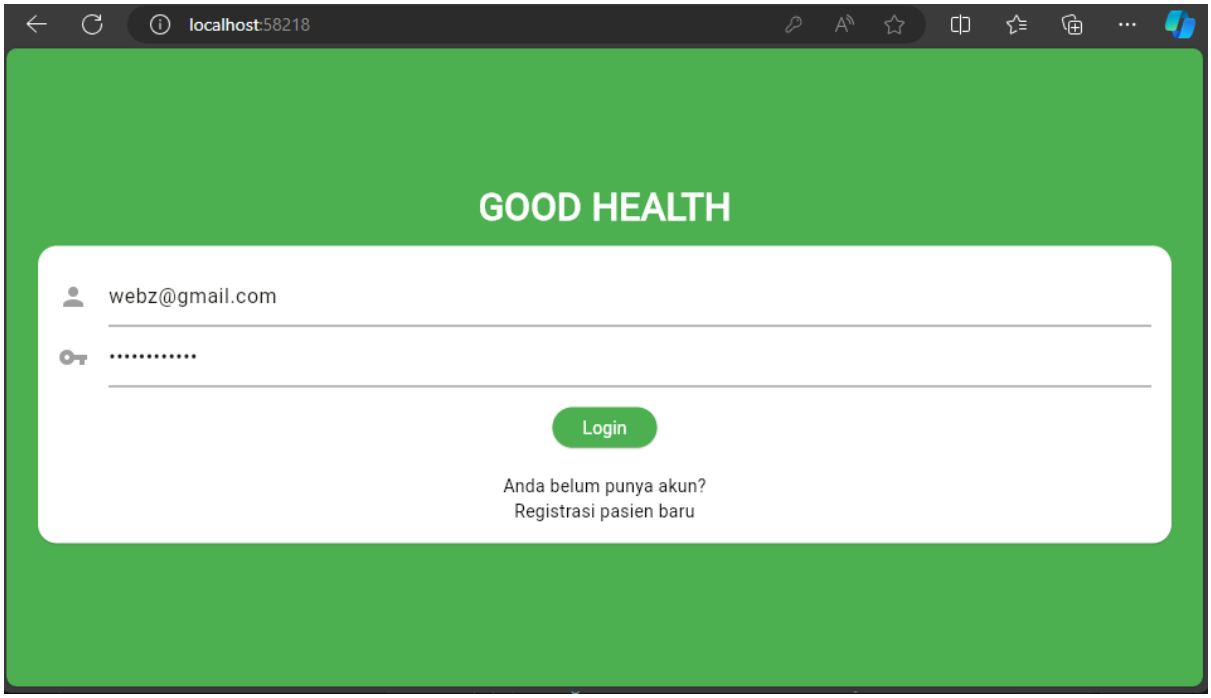
Tampilan register



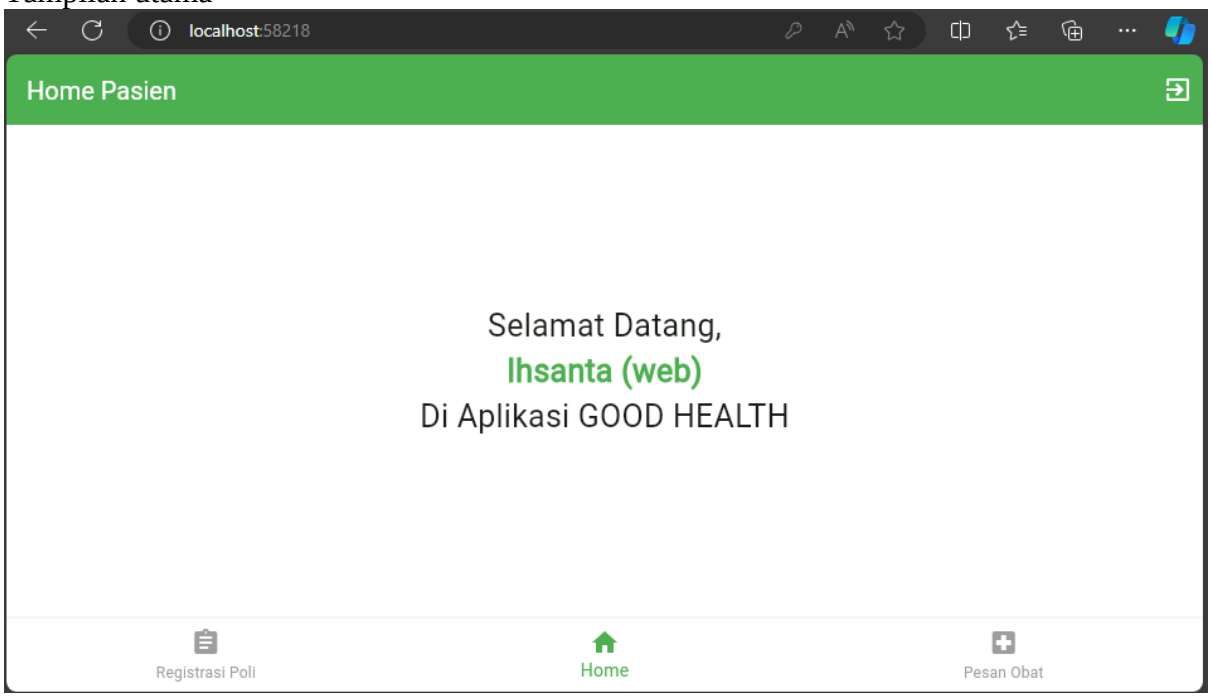
Tampilan berhasil register



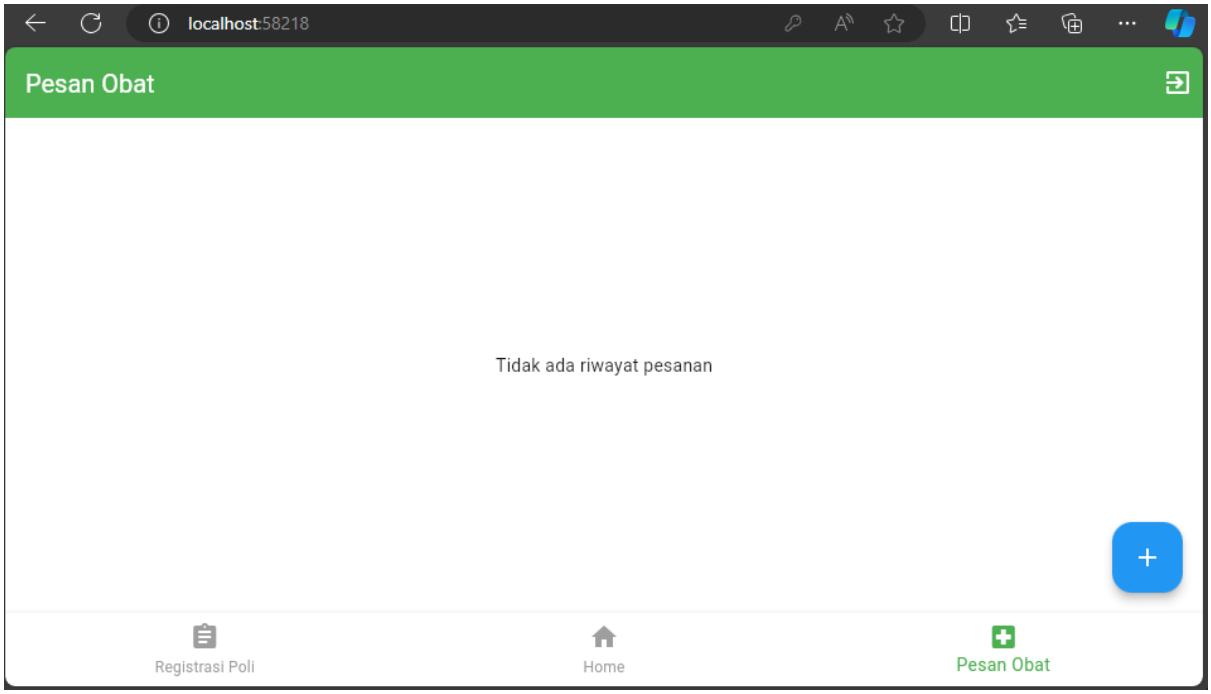
Tampilan login



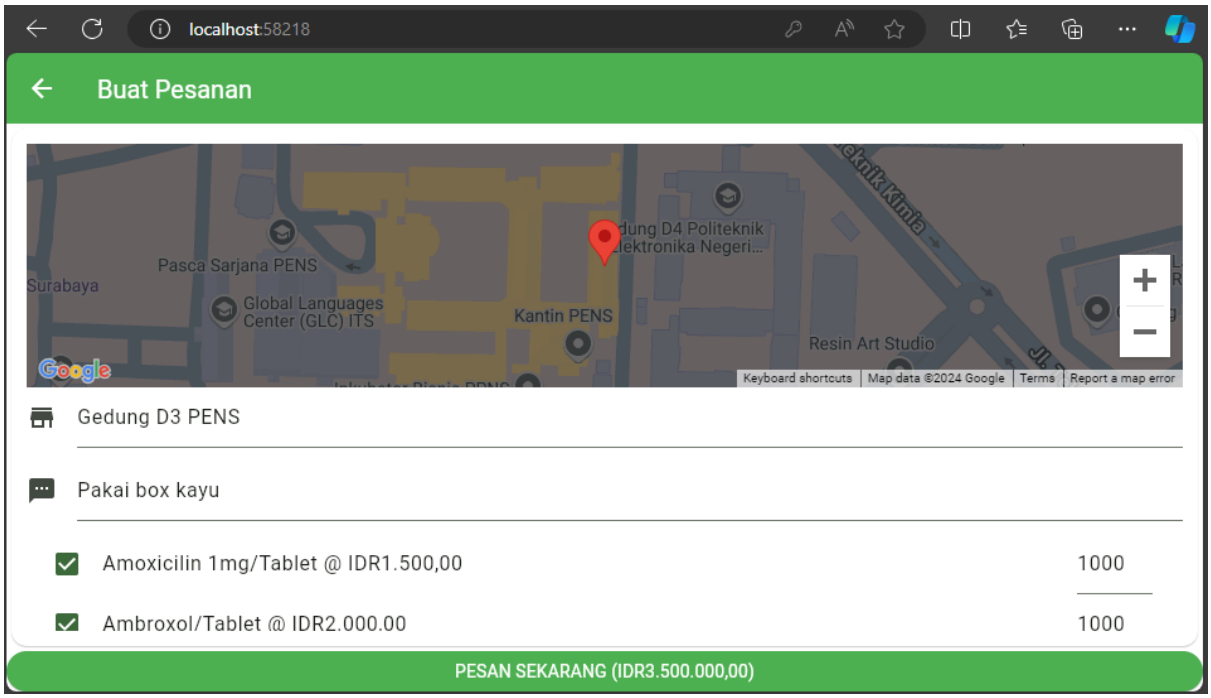
Tampilan utama



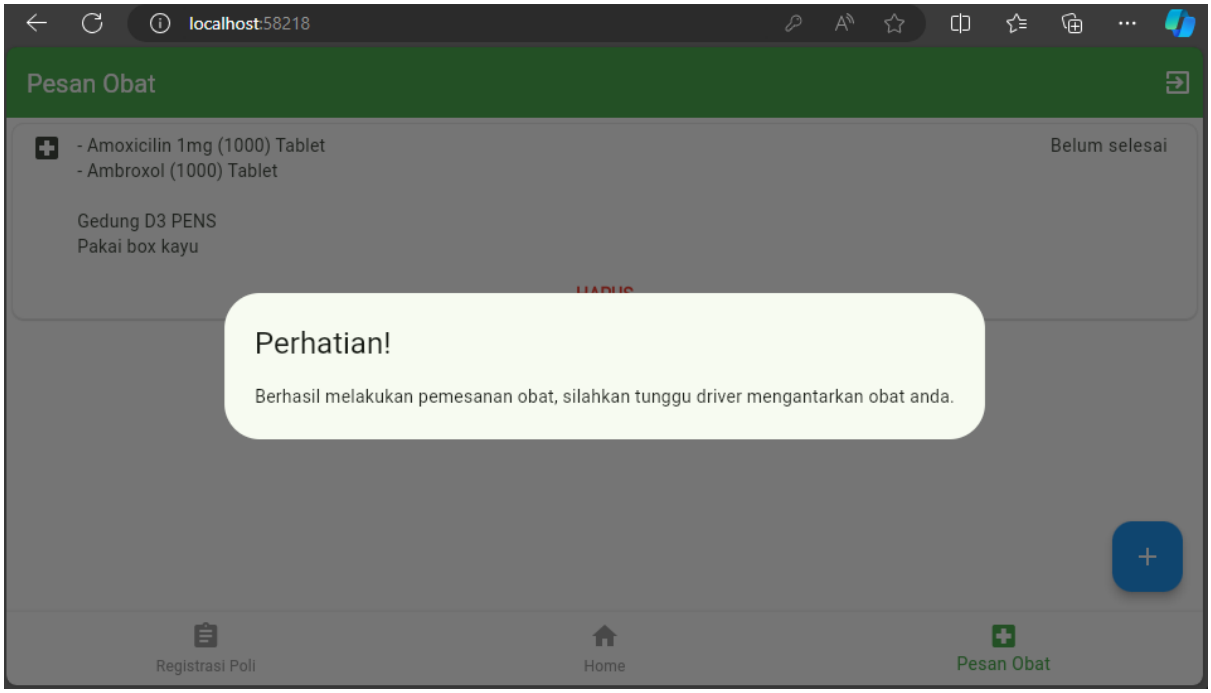
Tampilan pesan obat



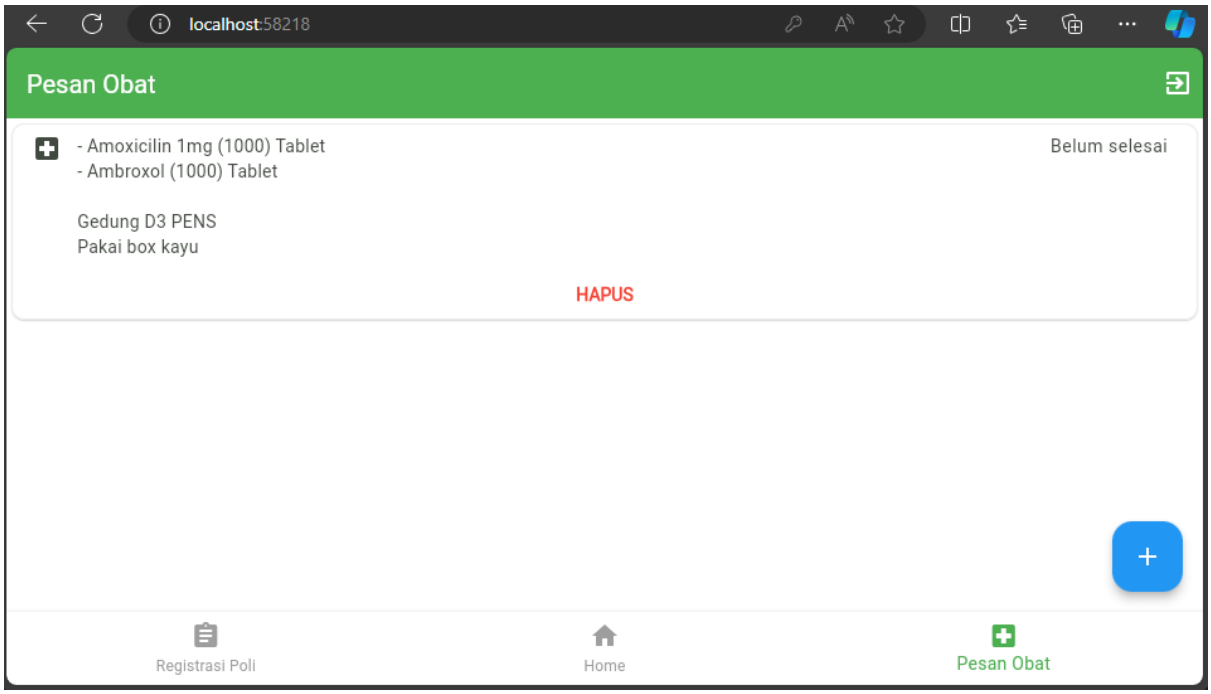
Tampilan menambah pesanan di “pesan obat”



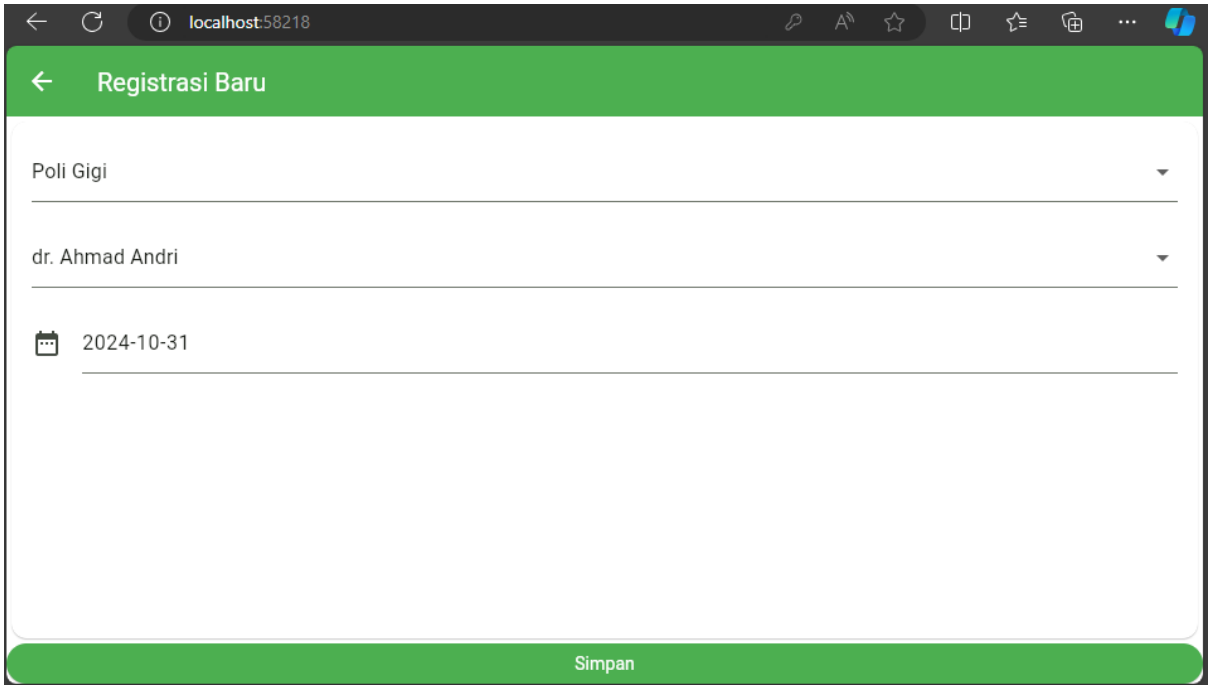
Tampilan notifikasi menambah pesanan berhasil



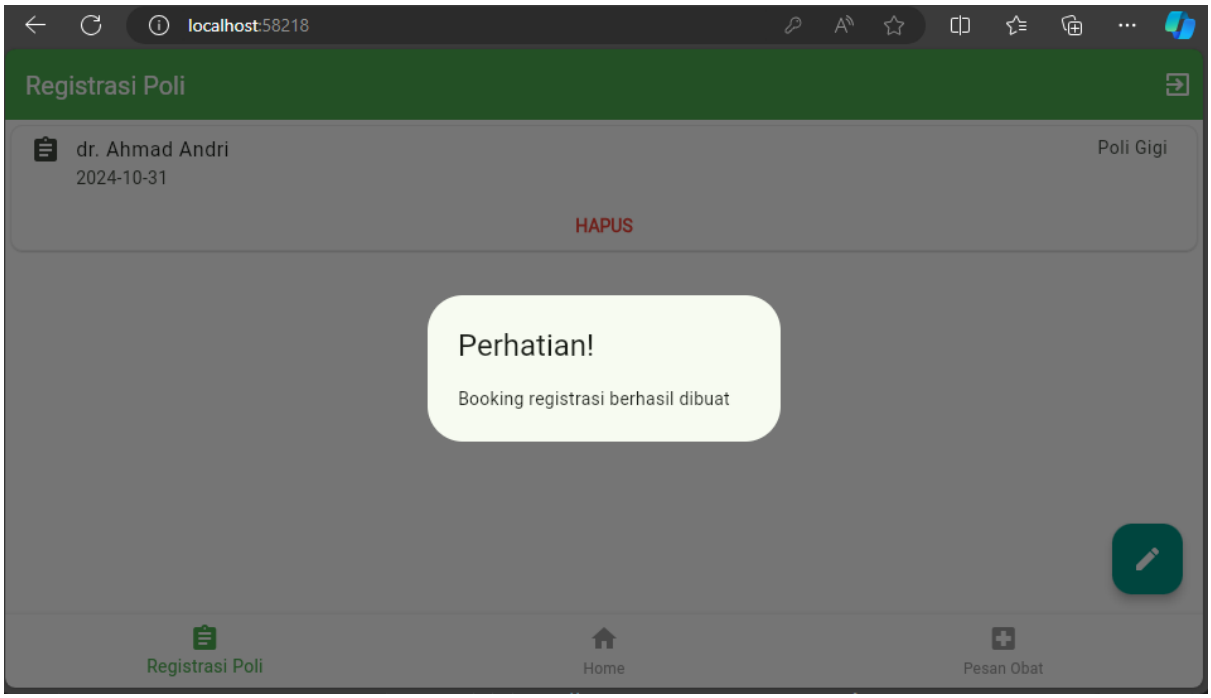
Tampilan pesanan tersedia



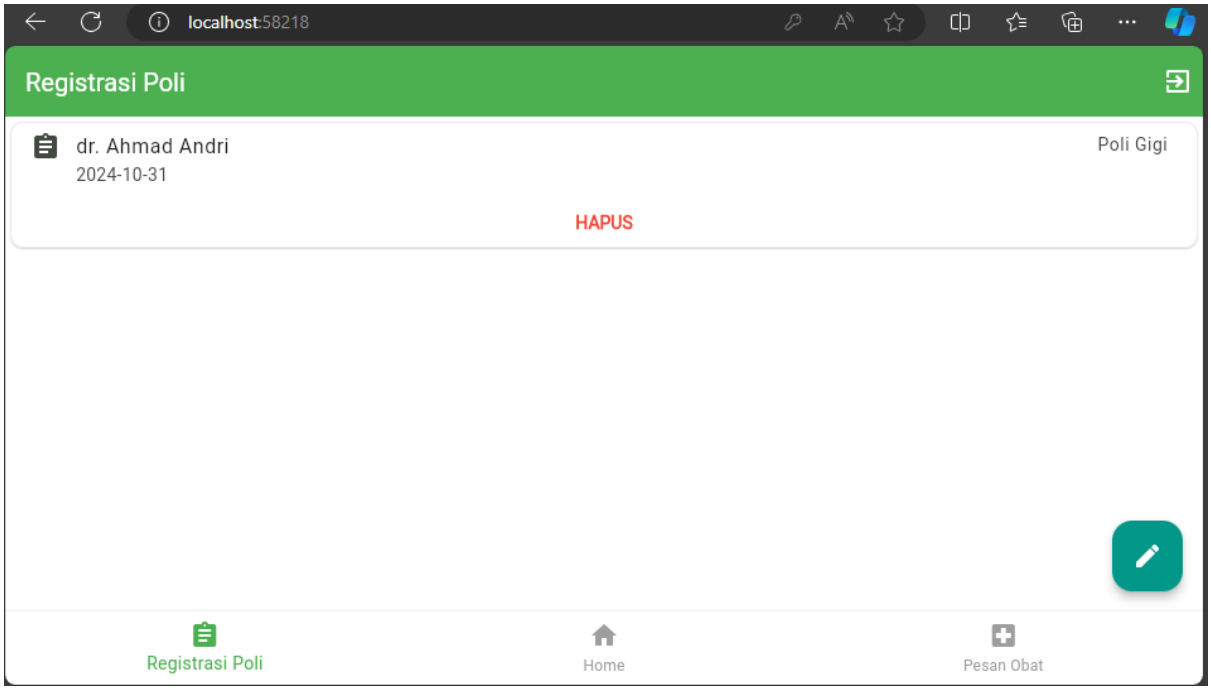
Tampilan melakukan registrasi di “registrasi poli”



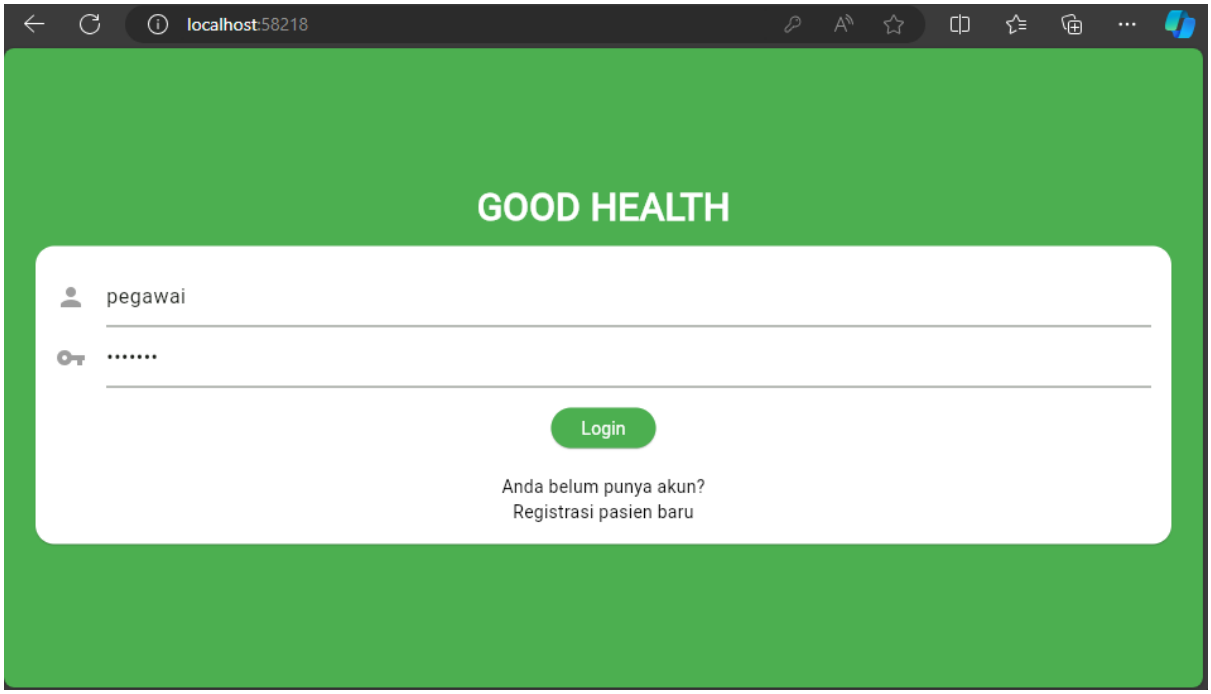
Tampilan notifikasi registrasi berhasil



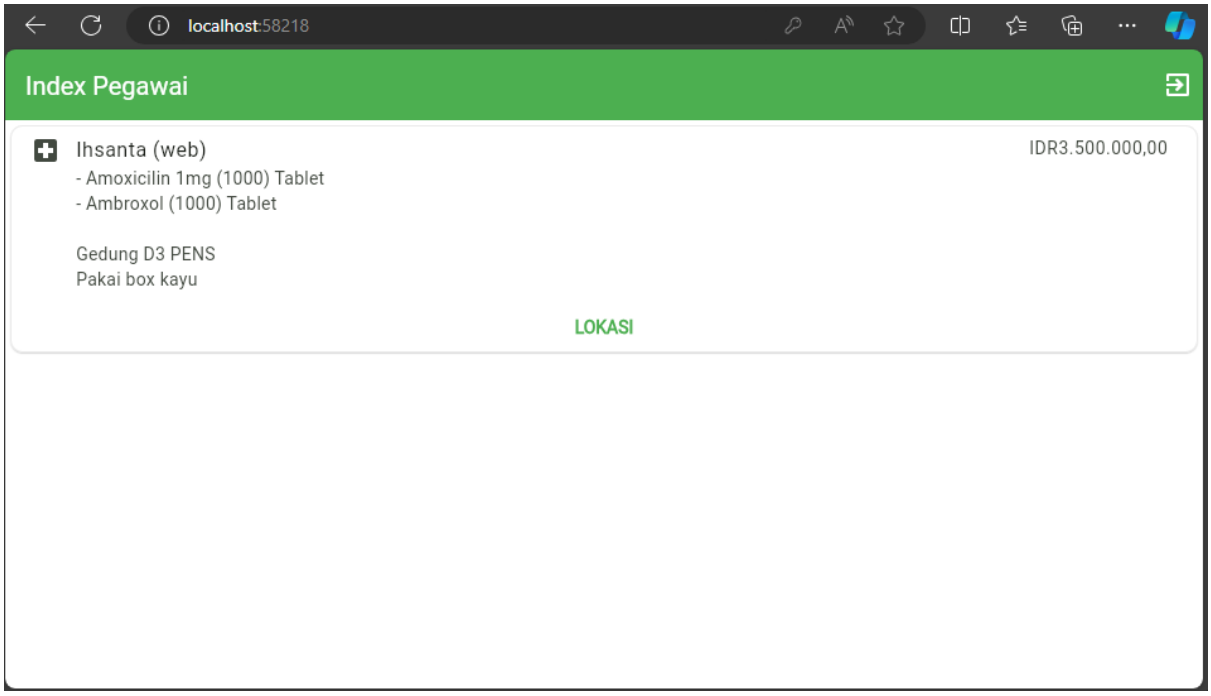
Tampilan registrasi berhasil ditambah



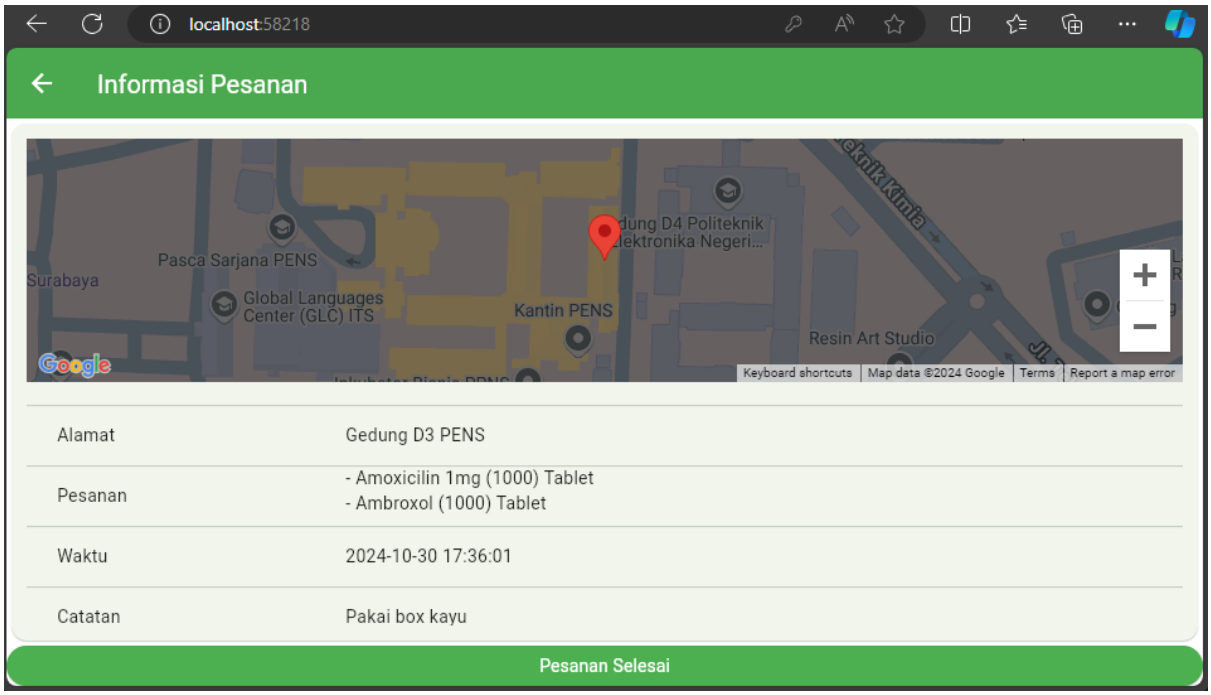
Tampilan login dengan akun pegawai username:pegawai, password:pegawai.



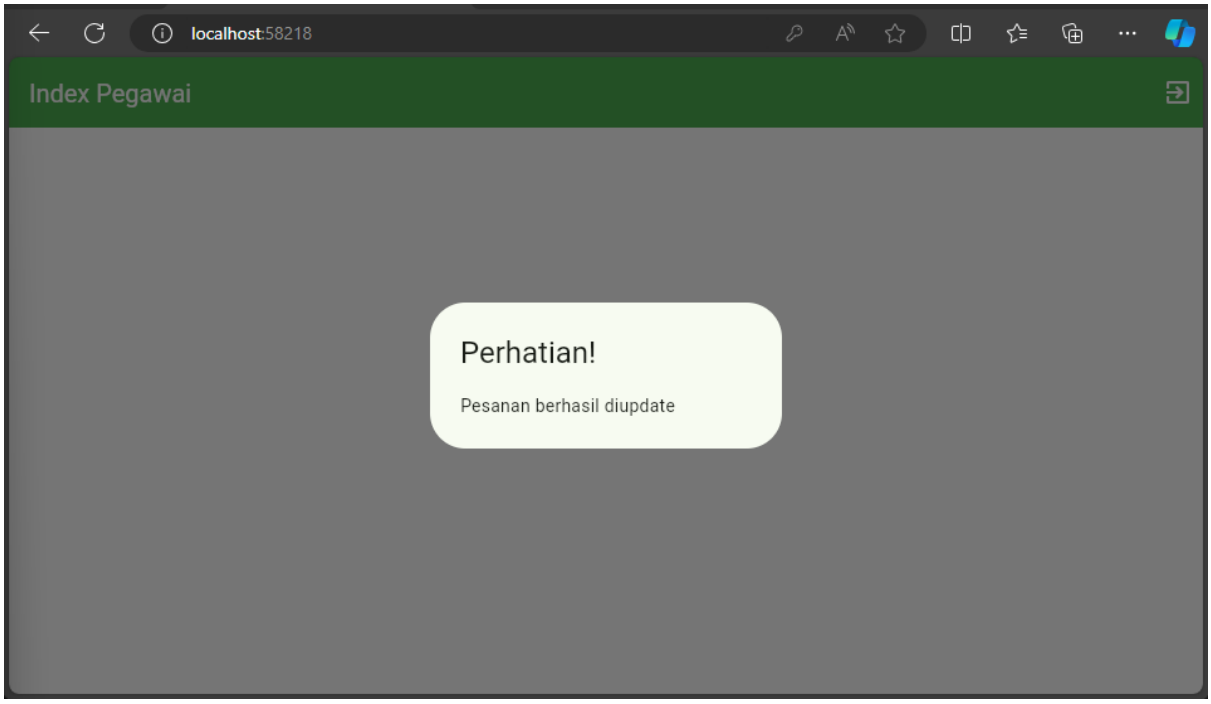
Tampilan login dengan akun pegawai berhasil



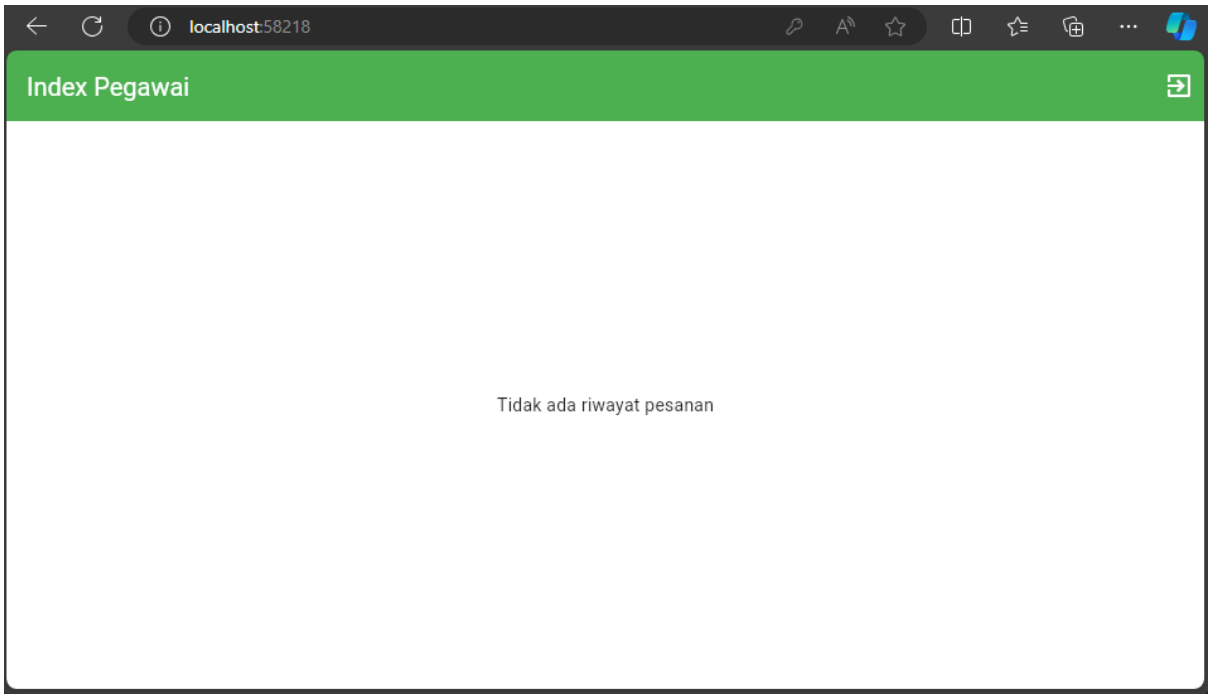
Tampilan informasi pesanan pada akun pegawai



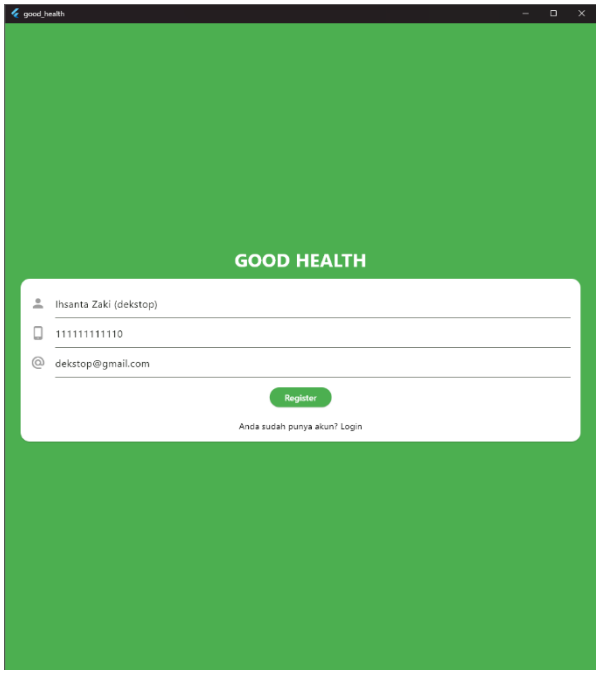
Tampilan notifikasi pesanan berhasil diselesaikan

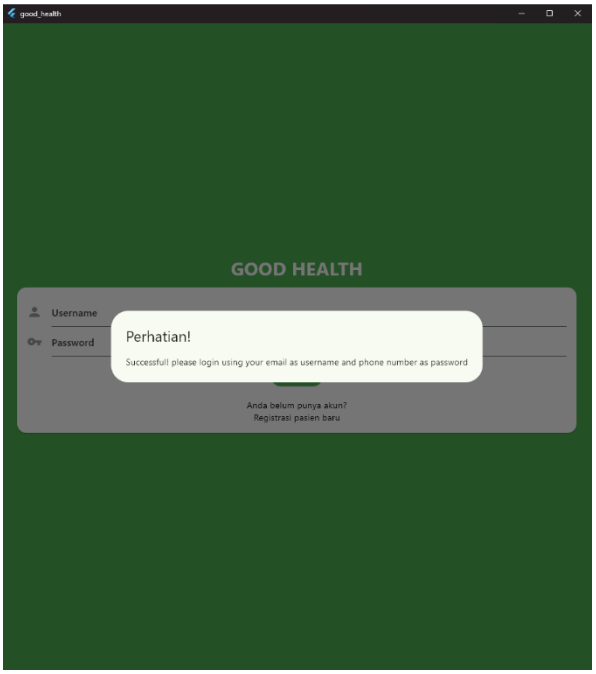


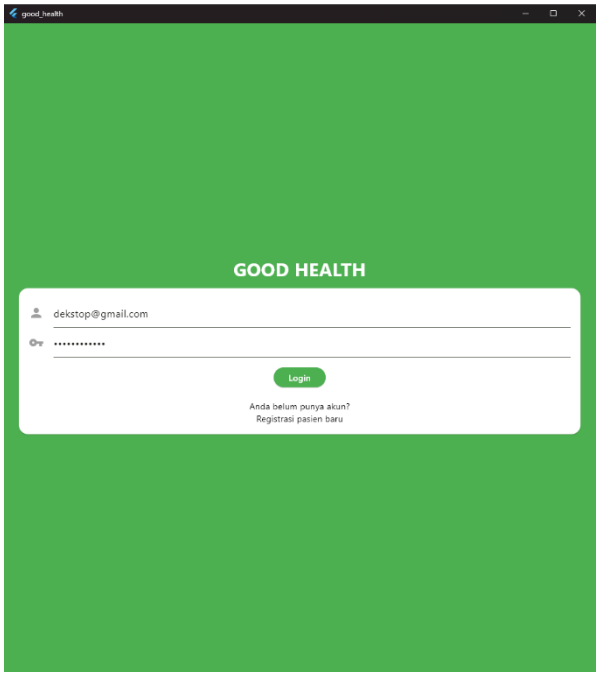
Tampilan pesanan kosong

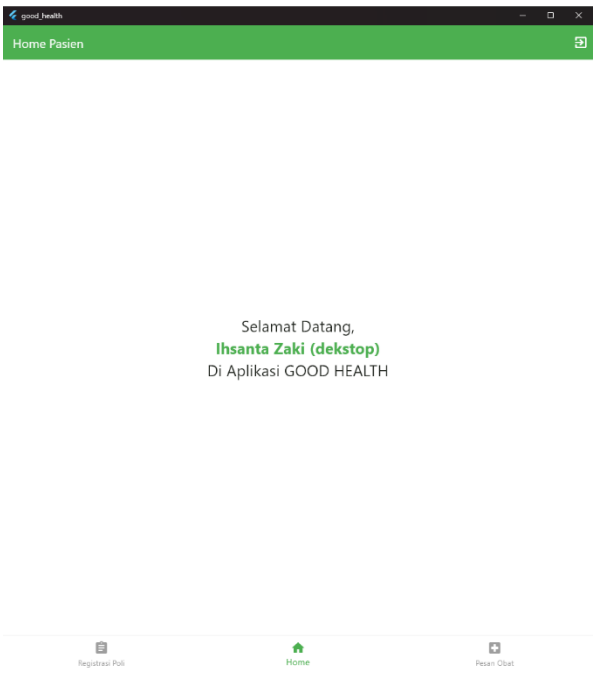


c. Tampilan pada platform Windows (dekstop)



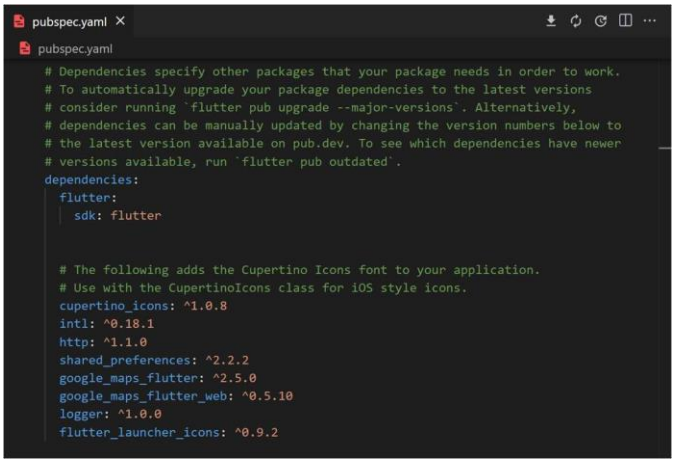






D. ANALISA

Pada praktikum ini, pengembangan aplikasi client-server berbasis mobile menggunakan Flutter dilakukan sebagai kelanjutan dari praktikum sebelumnya yang berfokus pada pembuatan REST API menggunakan PHP. Praktikum kali ini memperkenalkan konsep client-server, penerapan basis data, serta integrasi backend dengan Flutter yang memungkinkan aplikasi untuk mengakses dan mengelola data dari server secara real-time.



```yaml
# Dependencies specify other packages that your package needs in order to work.
# To automatically upgrade your package dependencies to the latest versions
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers below to
# the latest version available on pub.dev. To see which dependencies have newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter


  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  intl: ^0.18.1
  http: ^1.1.0
  shared_preferences: ^2.2.2
  google_maps_flutter: ^2.5.0
  google_maps_flutter_web: ^0.5.10
  logger: ^1.0.0
  flutter_launcher_icons: ^0.9.2
```

Beberapa dependency di pubspec.yaml memainkan peran penting dalam menghubungkan frontend dan backend serta menambahkan fitur-fitur pendukung. Dependency seperti http memungkinkan aplikasi untuk berkomunikasi dengan REST API, sementara shared_preferences membantu menyimpan data secara lokal, seperti sesi pengguna, yang mempercepat akses dan meningkatkan pengalaman pengguna. Selain itu, google_maps_flutter dan google_maps_flutter_web memungkinkan integrasi Google Maps, sehingga pengguna dapat mengakses peta dan fitur navigasi. Dependency tambahan seperti intl digunakan untuk format waktu dan data sesuai lokal, dan logger berguna untuk debugging

Pada tahap ini, aplikasi Flutter telah dikonfigurasi untuk memanfaatkan Google Maps API pada platform Android dan iOS dengan menambahkan API Key yang diperlukan ke dalam file konfigurasi masing-masing platform. Untuk Android, API Key didefinisikan dalam file AndroidManifest.xml yang terletak di direktori android/app/src/main/. File ini mengandung beberapa izin seperti INTERNET, ACCESS_NETWORK_STATE, dan izin lokasi (ACCESS_FINE_LOCATION dan ACCESS_COARSE_LOCATION) yang memungkinkan aplikasi mengakses jaringan dan data lokasi perangkat. Penambahan <meta-data> untuk com.google.android.geo.API_KEY berfungsi untuk menyediakan kunci API ke aplikasi, yang akan digunakan oleh plugin google_maps_flutter untuk memuat peta dalam aplikasi.

```
AndroidManifest.xml M
android > app > src > main > AndroidManifest.xml
You, 2 hours ago | 2 authors (rifqiraehan and one other)
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>

    <application
        android:label="Good Health"
        android:name="${applicationName}"
        android:icon="@mipmap/launcher_icon">

        <meta-data android:name="com.google.android.geo.API_KEY"
            android:value="AIzaSyA9uTVNAQLr5-25wlNq3Xvc2D00j_bWUK0"/>
```

Sedangkan pada platform iOS, konfigurasi dilakukan dengan menambahkan API Key pada file AppDelegate.swift di ios/Runner/Base.lproj/. Di sini, kode GMSServices.provideAPIKey digunakan untuk menginisialisasi layanan Google Maps dengan API Key yang sama sehingga aplikasi dapat menggunakan fitur peta di iOS.

```
AppDelegate.swift M
ios > Runner > AppDelegate.swift
You, 1 second ago | 2 authors (rifqiraehan and one other)
import Flutter
import UIKit
import GoogleMaps

@main
@objc class AppDelegate: FlutterAppDelegate {
  override func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
  ) -> Bool {
    GMSServices.provideAPIKey("AIzaSyA9uTVNAQLr5-25wlNq3Xvc2D00j_bWUK0")
    GeneratedPluginRegistrant.register(with: self)
    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
  }
}
```

Pengaturan pada file HTML menunjukkan bagaimana aplikasi web menggunakan Google Maps API secara asinkron, yang menambah kemampuan aplikasi ini untuk berjalan lintas platform dengan fitur peta yang konsisten.

```
index.html M
web > index.html > html > head > script
You, 2 hours ago | 2 authors (rifqiraehan and one other)
<!DOCTYPE html>
<html>
<head>
  <base href="$FLUTTER_BASE_HREF">

  <meta charset="UTF-8">
  <meta content="IE=Edge" http-equiv="X-UA-Compatible">
  <meta name="description" content="A new Flutter project.">

  <!-- iOS meta tags & icons -->
  <meta name="mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-status-bar-style" content="black">
  <meta name="apple-mobile-web-app-title" content="good_health">
  <link rel="apple-touch-icon" href="icons/Icon-192.png">

  <!-- Favicon -->
  <link rel="icon" type="image/png" href="favicon.ico"/>

  <title>Good Health</title>
  <link rel="manifest" href="manifest.json">

  <!-- Load Google Maps API asynchronously -->
  <script src="https://maps.googleapis.com/maps/api/js?key=AIzaSyA9uTVNAQLr5-25wlNq3Xvc2D00j_bWUK0" async></script>
</head>
<body>
  <script src="flutter_bootstrap.js" async></script>
</body>
</html>
```

Pengaturan yang terstruktur ini memastikan bahwa semua platform yang didukung oleh Flutter dapat mengakses API secara efisien.

Folder model akan dibuat di dalam lib, dengan enam file model inti yaitu dokter.dart, obat.dart, pasien.dart, pesan_obat.dart, reg_poli.dart, dan user.dart. Setiap file akan mendefinisikan kelas data yang sesuai dan mendefinisikan atribut seperti berikut:

- dokter.dart
  Model Dokter mendefinisikan atribut dasar dokter seperti idDokter, nama, dan hp. Model ini juga menyertakan fungsi fromJson untuk konversi JSON ke objek Dart, yang akan mempermudah pemrosesan data API. Fungsi fetchDokters melakukan permintaan GET untuk mengambil data dokter dari server, serta menangani respons dan konversi data.
- obat.dart
  Model Obat mencakup atribut seperti idObat, nama, harga, satuan, dan properti untuk status pemilihan obat dalam daftar. Fungsi obatFromJson mengonversi data JSON menjadi daftar objek Obat, sedangkan fetchObats menangani permintaan GET dan memeriksa konsistensi data respons, yang ditunjang oleh Logger untuk menangani log error.
- pasien.dart
  Model Pasien mendefinisikan atribut pasien, termasuk idPasien, nama, hp, dan email. Fungsi pasienCreate memungkinkan registrasi pasien baru ke sistem melalui permintaan POST, serta mengelola respons dan log error jika terjadi masalah. Dengan demikian, model ini memfasilitasi pengelolaan data pasien di seluruh aplikasi.
- pesan_obat.dart
  Model PesanObat mencakup atribut lengkap untuk pesanan obat seperti alamat, lat, lng, dan totalBiaya, serta mendukung pemrosesan status pesanan. Fungsi fetchPesanObats memungkinkan pengambilan daftar pesanan berdasarkan ID pasien, sementara fungsi pesanObatCreate memfasilitasi pembuatan pesanan baru ke server.
- reg_poli.dart
  Model RegisPoli mendefinisikan pendaftaran poli dengan atribut seperti idRegisPoli, tglBooking, dan poli, serta keterkaitan dengan objek Pasien dan Dokter. Fungsi fetchRegisPolis mengelola permintaan GET untuk daftar pendaftaran poli, sedangkan regisPoliCreate menangani pendaftaran baru dengan permintaan POST.
- user.dart
  Model User mendefinisikan atribut pengguna seperti username, password, idUser, dan idPasien, dengan fungsi login untuk proses autentikasi. Fungsi ini melakukan permintaan POST dan mengelola log untuk memeriksa status autentikasi, menyediakan antarmuka autentikasi pengguna yang penting bagi aplikasi.

Folder util ditambahkan untuk memisahkan fungsi pendukung dan pengelolaan sesi dari fungsi utama, dengan dua file utama: session.dart untuk pengaturan sesi pengguna dan util.dart untuk berbagai utilitas aplikasi. File session.dart menangani logika manajemen sesi dengan menggunakan SharedPreferences untuk menyimpan status login, jenis pengguna, dan informasi terkait pasien. Fungsi-fungsi seperti createPasienSession dan createPegawaiSession menyediakan metode yang efisien untuk menyimpan data pengguna selama sesi aktif, dan clearSession menghapus data saat logout.

```
import 'package:shared_preferences/shared_preferences.dart';
import 'package:good_health/model/pasien.dart';

const IS_LOGIN = 'IS_LOGIN';
const JENIS_LOGIN = 'JENIS_LOGIN';
const ID_PASIEN = 'ID_PASIEN';
const NAMA = 'NAMA';
const HP = 'HP';
const EMAIL = 'EMAIL';

enum jenisLogin { PASIEN, PEGAWAI }

Future createPasienSession(Pasien pasien) async {
  final prefs = await SharedPreferences.getInstance();
  prefs.setBool(IS_LOGIN, true);
  prefs.setString(ID_PASIEN, pasien.idPasien);
  prefs.setString(NAMA, pasien.nama);
  prefs.setString(HP, pasien.hp);
  prefs.setString(EMAIL, pasien.email);
  prefs.setString(JENIS_LOGIN, jenisLogin.PASIEN.toString());
  return true;
}

Future createPasienSessionx(String id, String nama, String hp, String email) async {
  final prefs = await SharedPreferences.getInstance();
  prefs.setBool(IS_LOGIN, true);
  prefs.setString(ID_PASIEN, id);
  prefs.setString(NAMA, nama);
  prefs.setString(HP, hp);
  prefs.setString(EMAIL, email);
  prefs.setString(JENIS_LOGIN, jenisLogin.PASIEN.toString());
  return true;
}

Future createPegawaiSession(String username) async {
  final prefs = await SharedPreferences.getInstance();
  prefs.setBool(IS_LOGIN, true);
  prefs.setString(NAMA, username);
  prefs.setString(JENIS_LOGIN, jenisLogin.PEGAWAI.toString());
  return true;
}

Future clearSession() async {
  final prefs = await SharedPreferences.getInstance();
  prefs.clear();
  return true;
}
```

Sementara itu, util.dart mendukung komponen tampilan dan berbagai fungsi umum, termasuk dialog untuk menampilkan pesan dialog dan largetButton sebagai template tombol berukuran besar di bagian bawah layar. Fungsi toRupiah mengubah nilai integer menjadi format mata uang Rupiah, mendukung pengalaman pengguna dengan tampilan nilai yang lebih mudah dibaca. Fungsi logOut mengelola alur logout dengan transisi yang halus ke halaman login setelah sesi dihapus, membuat alur pengguna menjadi lebih intuitif.

```
import 'package:flutter/material.dart';
import 'package:intl/intl.dart';
import 'package:good_health/page/login.dart';
import 'package:good_health/util/session.dart';

dialog(context, msg, {title}) {
  showDialog(
```

```
    context: context,
    builder: (BuildContext context) {
      return AlertDialog(
        title: Text(title ?? 'Perhatian!'),
        content: Text(msg.toString()),
      );
    },
  );
}

// Template button lebar utk posisi bottomNavigationBar
Widget largetButton(
    {String label = "Simpan", IconData? iconData, Function? onPressed}) {
  iconData = iconData ?? Icons.done_all;
  return SizedBox(
    height: 60,
    width: double.infinity,
    child: ElevatedButton.icon(
        style: ElevatedButton.styleFrom(
          elevation: 4.0,
          backgroundColor: Colors.blue,
          disabledForegroundColor: Colors.grey.withOpacity(0.38),
          disabledBackgroundColor: Colors.grey.withOpacity(0.12),
        ),
        label: Text(
          label,
          style: const TextStyle(
            color: Colors.white,
            fontSize: 16.0,
          ),
        ),
        icon: Icon(iconData, color: Colors.white),
        onPressed: () {}),
  );
}

// fungsi format tulisan rupiah
String toRupiah(int val) {
  final formatter = NumberFormat.currency(locale: 'ID');
  return formatter.format(val);
}

void logOut(BuildContext context) {
  clearSession().then((value) {
    if (!context.mounted) return;
    Navigator.pushAndRemoveUntil(
      context,
      PageRouteBuilder(pageBuilder: (BuildContext context, Animation animation,
          Animation secondaryAnimation) {
        return const LoginPage();
      }, transitionsBuilder: (BuildContext context, Animation<double> animation,
          Animation<double> secondaryAnimation, Widget child) {
        return SlideTransition(
          position: Tween<Offset>(
            begin: const Offset(1.0, 0.0),
            end: Offset.zero,
          ).animate(animation),
          child: child,
        );
      }),
      (route) => false,
    );
  });
```

Kemudian, modifikasi file main.dart dilakukan untuk mengatur alur utama aplikasi dan

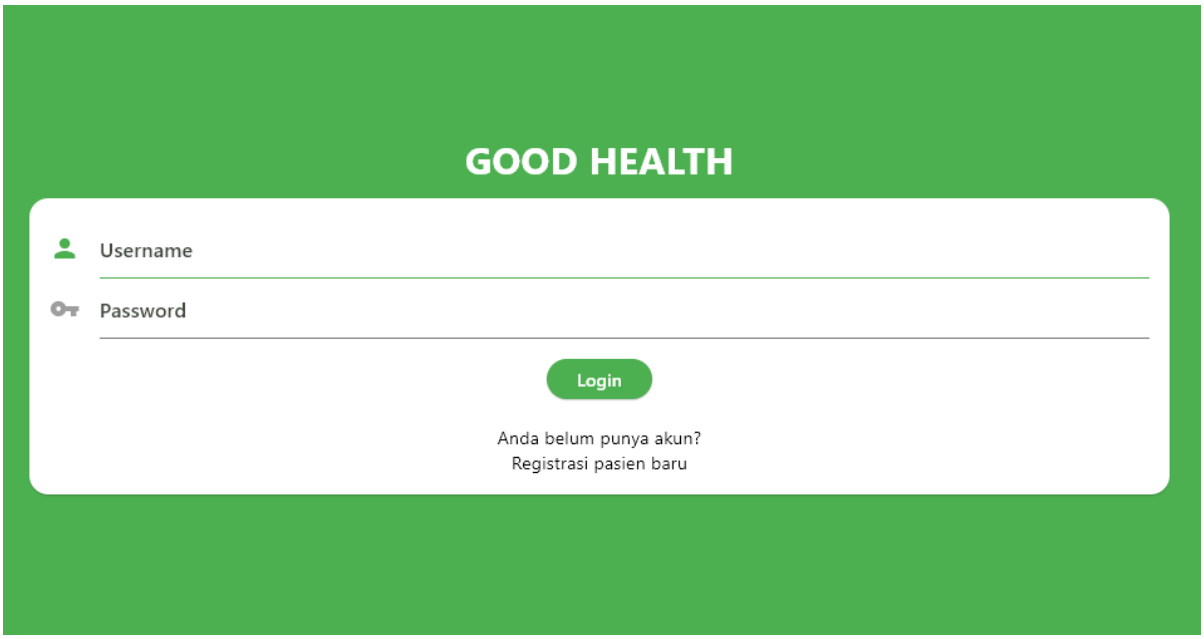
mengarahkan pengguna ke halaman yang sesuai berdasarkan status sesi mereka. Aplikasi dimulai dari fungsi main(), yang menjalankan widget utama MyApp. MyApp, yang kini dikelola oleh MyAppState, mengimplementasikan tema utama aplikasi dengan warna dominan hijau, dari tombol hingga bilah navigasi bawah, untuk tampilan visual yang serasi dan khas. Di dalam build() pada MyAppState, aplikasi menggunakan FutureBuilder untuk memuat status sesi pengguna dari SharedPreferences, yang mengidentifikasi apakah pengguna sudah login atau belum. Ini dilakukan dengan metode asinkron _loadSession() yang mengakses data login pengguna.

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:good_health/page/login.dart';
import 'package:good_health/page/modul_pasien/index.dart' as index_pasien;
import 'package:good_health/page/modul_pegawai/index.dart' as index_pegawai;
import 'package:good_health/util/session.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatefulWidget {
  const MyApp({super.key});

  @override
  MyAppState createState() => MyAppState();
}

class MyAppState extends State<MyApp> {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primaryColor: Colors.green,
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
        primarySwatch: Colors.green,
        scaffoldBackgroundColor: Colors.white,
        appBarTheme: const AppBarTheme(
          backgroundColor: Colors.green,
          foregroundColor: Colors.white,
          titleTextStyle: TextStyle(
            color: Colors.white,
            fontSize: 20,
          ),
          iconTheme: IconThemeData(
            color: Colors.white, //
          ),
        ),
        bottomNavigationBarTheme: const BottomNavigationBarThemeData(
          backgroundColor: Colors.white,
          selectedItemColor: Colors.green,
          unselectedItemColor: Colors.grey,
        ),
        elevatedButtonTheme: ElevatedButtonThemeData(
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.green,
            foregroundColor: Colors.white,
          ),
        ),
      ),
      home: FutureBuilder(
        future: _loadSession(),
        builder: (context, snapshot) {
          late final SharedPreferences prefs = snapshot.data;
```

```
          late Widget result;
          if (snapshot.connectionState == ConnectionState.waiting) {
            result = const Scaffold(
              body: Center(
                child: CircularProgressIndicator(),
              ),
            );
          } else if (snapshot.connectionState == ConnectionState.done) {
            if (snapshot.hasData) {
              if (prefs.getBool(IS_LOGIN) ?? false) {
                if (prefs.getString(JENIS_LOGIN) ==
                    jenisLogin.PASIEN.toString()) {
                  result = const index_pasien.IndexPage();
                } else {
                  result = const index_pegawai.IndexPage();
                }
              } else {
                result = const LoginPage();
              }
            } else {
              return const Center(child: Text('Error..'));
            }
          }
          return result;
        },
      ),
    );
  }

  Future _loadSession() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs;
  }
}
```

Saat FutureBuilder mendeteksi bahwa data sesi sudah siap, widget yang tepat akan ditampilkan tergantung pada status login dan jenis pengguna. Jika pengguna adalah pasien, halaman index_pasien.IndexPage() ditampilkan; jika pegawai, maka halaman index_pegawai.IndexPage() yang akan terbuka. Bagi pengguna yang belum login, aplikasi akan menampilkan LoginPage sebagai halaman default

Untuk membuat antarmuka login, buat folder baru bernama page dan file login.dart di dalamnya. Dalam file ini, LoginPage didefinisikan sebagai widget stateful yang menangani logika dan tampilan form login. Antarmuka memiliki latar belakang hijau dengan teks judul yang mencolok, diikuti oleh kartu berisi form login yang dirancang untuk memudahkan pengguna dalam memasukkan nama pengguna dan kata sandi. Terdapat validasi sederhana pada kedua field input, yang memastikan bahwa tidak ada data kosong yang dapat dikirimkan. Selain itu, interaksi pengguna ditingkatkan dengan fitur hover pada teks yang mengarahkan pengguna untuk registrasi jika mereka belum memiliki akun.



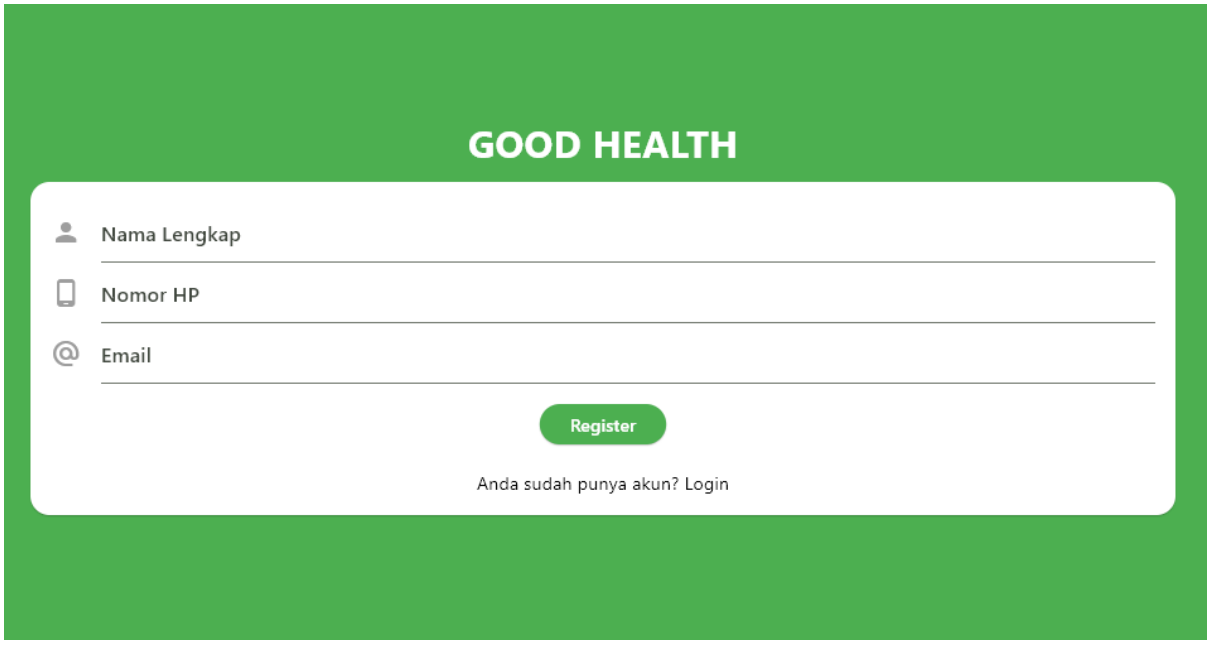
Laman Login

Logika login dikendalikan melalui fungsi _login(), yang memvalidasi input dan kemudian mengirimkan permintaan ke server untuk memverifikasi kredensial pengguna. Jika login berhasil, sesi pengguna akan dibuat menggunakan createPasienSessionx atau createPegawaiSession, tergantung pada jenis pengguna yang terdaftar. Aplikasi kemudian akan mengalihkan pengguna ke halaman yang sesuai, baik untuk pasien atau pegawai, berdasarkan informasi yang diterima dari server. Jika terjadi kesalahan, pengguna akan diberi umpan balik melalui dialog yang menunjukkan pesan kesalahan yang relevan.

Validasi pada Laman Login

Untuk membuat antarmuka pendaftaran pengguna, langkah selanjutnya adalah membuat folder modul_pasien dan file baru bernama signup.dart, yang berisi kelas SignUpPage sebagai widget

stateful. Halaman ini memiliki desain serupa dengan halaman login, dengan latar belakang hijau dan teks judul yang menonjol, serta form yang memungkinkan pengguna memasukkan nama lengkap, nomor telepon, dan alamat email, lengkap dengan validasi untuk memastikan semua field terisi. Fungsi prosesRegistrasi() mengelola logika pendaftaran dengan mengirimkan data ke server, dan jika berhasil, respons akan ditampilkan kepada pengguna. Logger juga ditambahkan untuk mencatat informasi respons, sehingga memudahkan debugging jika terjadi kesalahan.



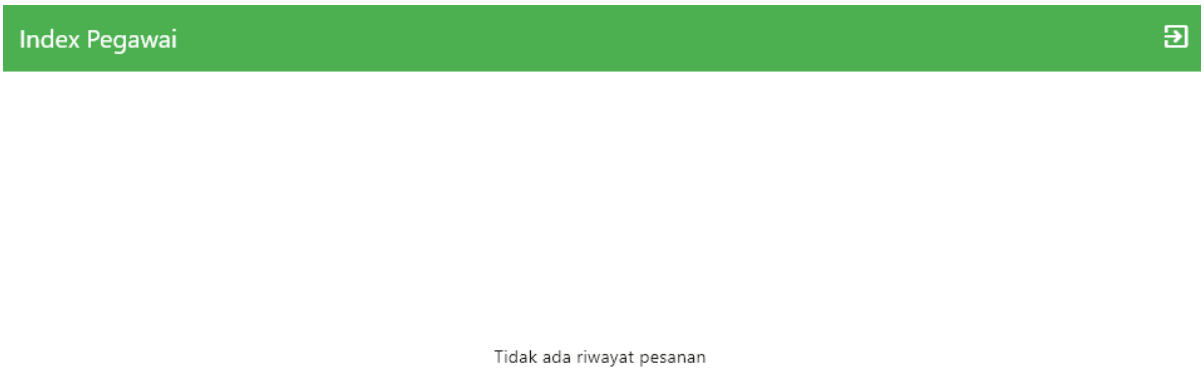
Laman Register

Dalam pengembangan aplikasi, pengguna pasien akan diberikan tiga menu utama: Home, Registrasi Poli, dan Pesan Obat. Untuk menyusun struktur ini, tiga folder baru akan dibuat, yaitu reg_poli, home, dan modul_pasien. Di dalam folder modul_pasien, file index.dart akan diciptakan, yang berfungsi sebagai halaman utama untuk menampilkan konten berdasarkan menu yang dipilih. Dengan menggunakan widget BottomNavigationBar, aplikasi akan memungkinkan pengguna untuk berpindah antara tampilan yang berbeda dengan mudah, dan setiap tab akan memperlihatkan konten spesifik dari masing-masing halaman.

Bottom Navigator Bar

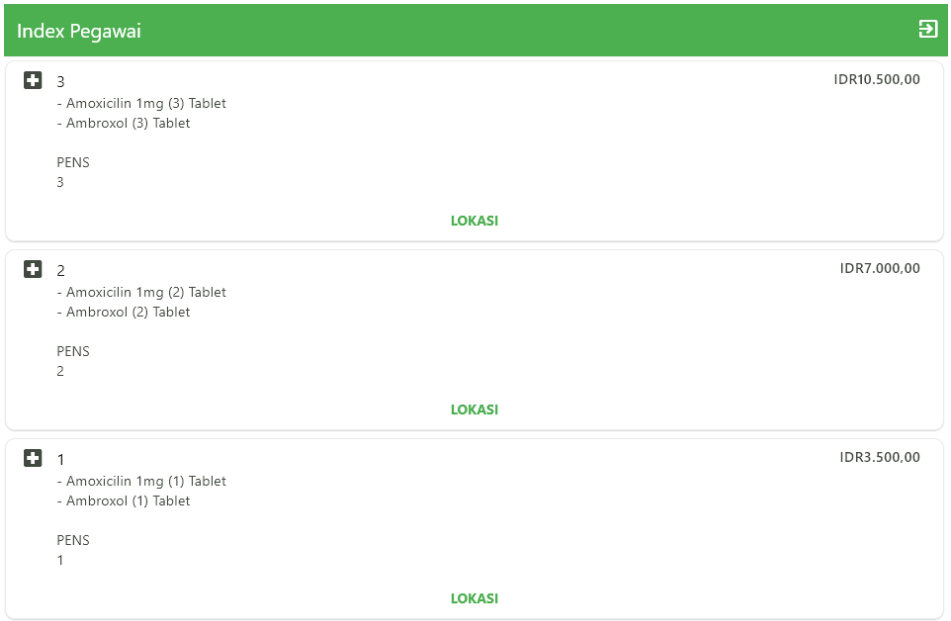
Selain pengguna pasien, aplikasi ini juga akan melayani pegawai. Untuk mendukung kebutuhan ini, folder baru bernama modul_pegawai akan dibuat, di dalamnya terdapat file index.dart. Halaman ini akan dirancang untuk menampilkan informasi yang relevan bagi pegawai, seperti daftar pesan obat yang masuk. Dalam implementasinya, data akan diambil menggunakan FutureBuilder, sehingga pegawai dapat melihat dan mengelola informasi

dengan efisien. Dengan pendekatan ini, aplikasi akan memberikan antarmuka yang terpisah namun terintegrasi.



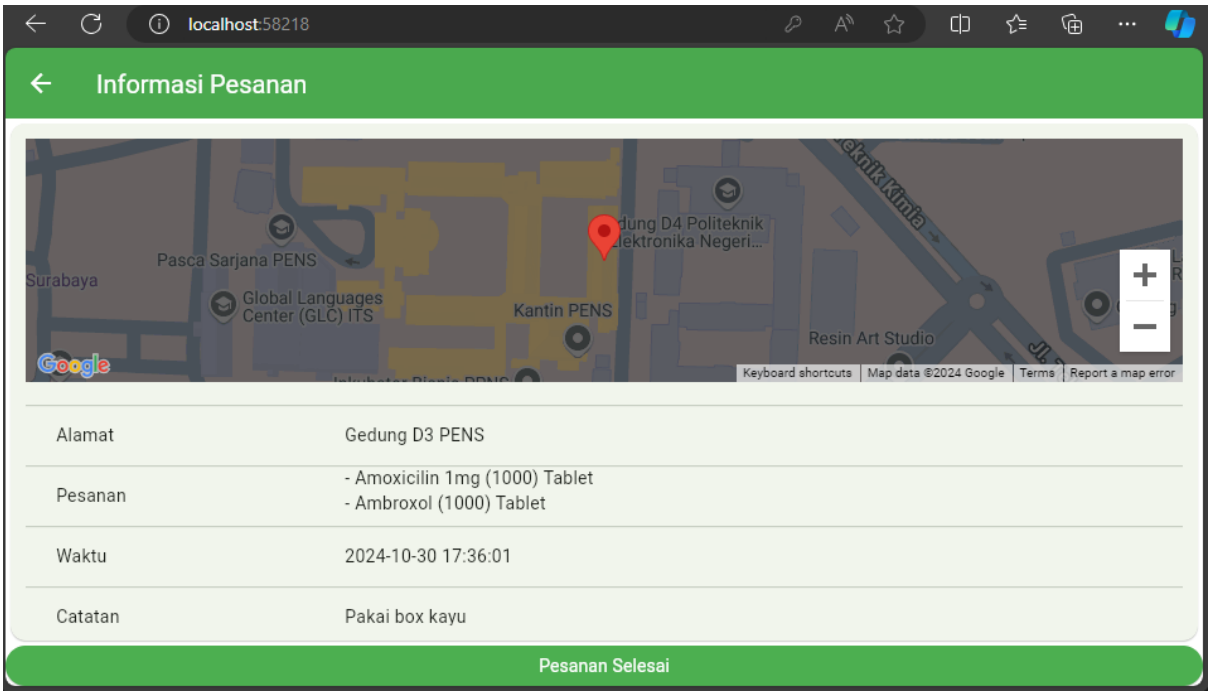
Laman Utama dengan user pegawai

Untuk menyimpan dan menampilkan data secara efisien, folder baru bernama list_widget akan dibuat di dalam folder page. Di sini, file pesan_obat_pegawai.dart, pesan_obat.dart, dan reg_poli.dart akan dibuat untuk mengelola tampilan daftar sesuai dengan konteksnya. Setiap widget akan dirancang untuk menampilkan daftar entri yang relevan, menggunakan ListView.builder untuk pengoptimalan performa. Penggunaan Card dan ListTile dalam tampilan akan memberikan pengalaman visual yang bersih dan mudah digunakan, memungkinkan pengguna untuk berinteraksi dengan informasi yang ditampilkan.



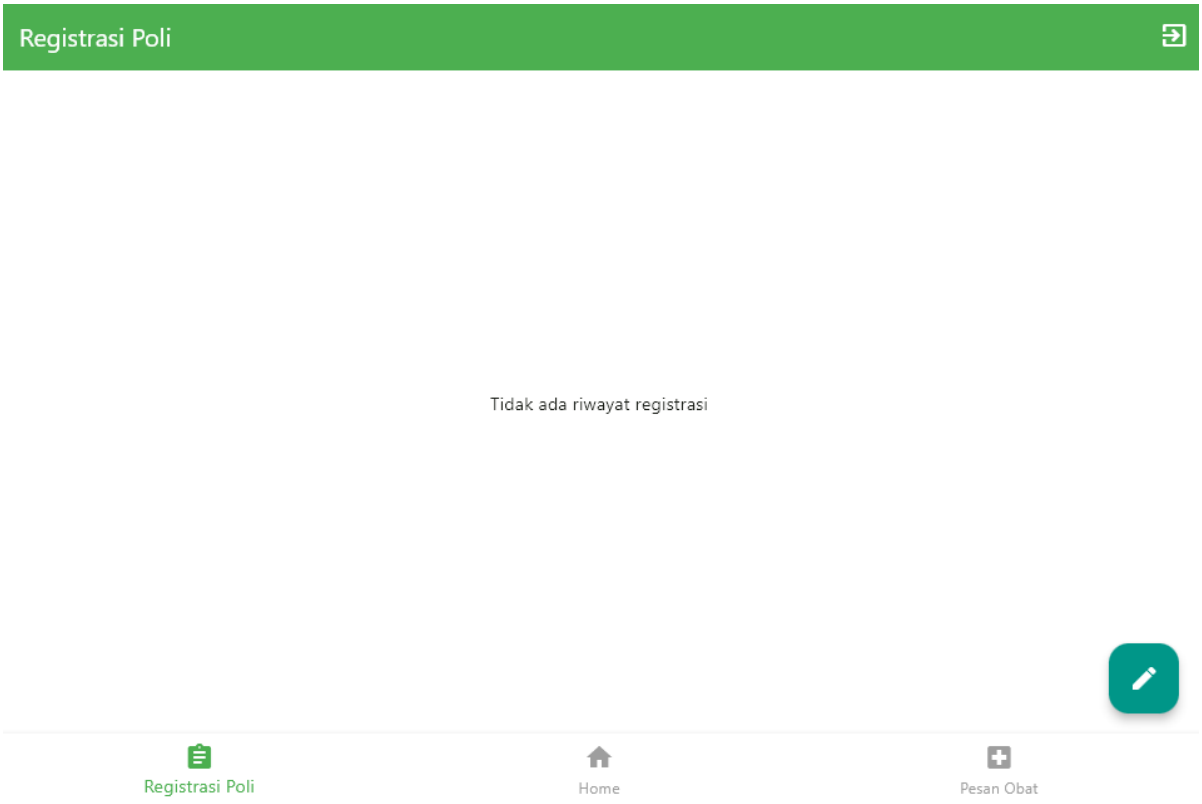
Daftar Entri Pesanan pada tampilan dengan user pegawai

Di dalam folder modul_pegawai, file baru dengan nama pesan_obat_view.dart akan dikembangkan untuk memberikan detail lebih lanjut mengenai setiap pesanan obat. Halaman ini akan menampilkan informasi seperti nama pemesan, nomor telepon, alamat, serta daftar pesanan dalam format tabel menggunakan DataTable. Selain itu, pengguna akan diberikan tampilan peta interaktif menggunakan GoogleMap untuk menunjukkan lokasi terkait. Fitur ini akan memudahkan pegawai dalam melakukan pengecekan dan verifikasi pesanan secara real-time, serta memungkinkan tindakan lebih lanjut seperti memperbarui status pesanan dengan menggunakan tombol.



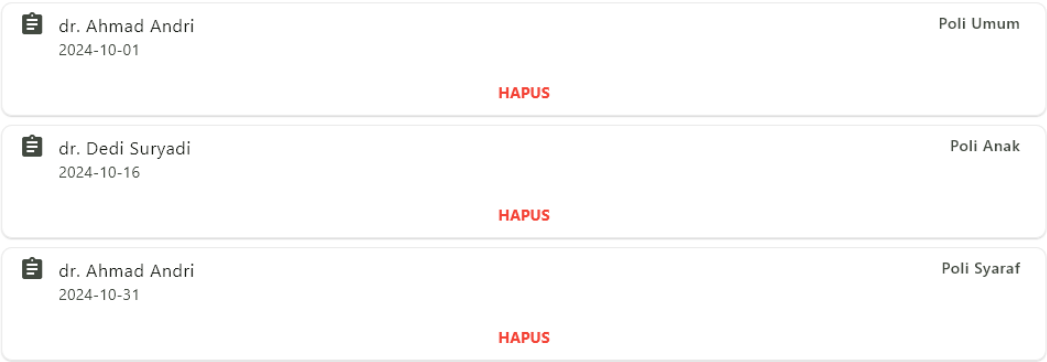
Tampilan informasi pesanan pada akun pegawai

Reg_poli_content adalah bagian penting dari fitur registrasi poli. Dengan menggunakan StatefulWidget, RegisPoliContent mengelola status dan memuat daftar registrasi poli secara asinkron dengan memanfaatkan FutureBuilder. FloatingActionButton diimplementasikan untuk memberikan opsi bagi pengguna untuk menambahkan registrasi baru. Kode ini juga menekankan pentingnya pengecekan kondisi mounted, memastikan bahwa widget masih aktif saat melakukan pembaruan state setelah pengguna kembali dari halaman pembuatan registrasi.



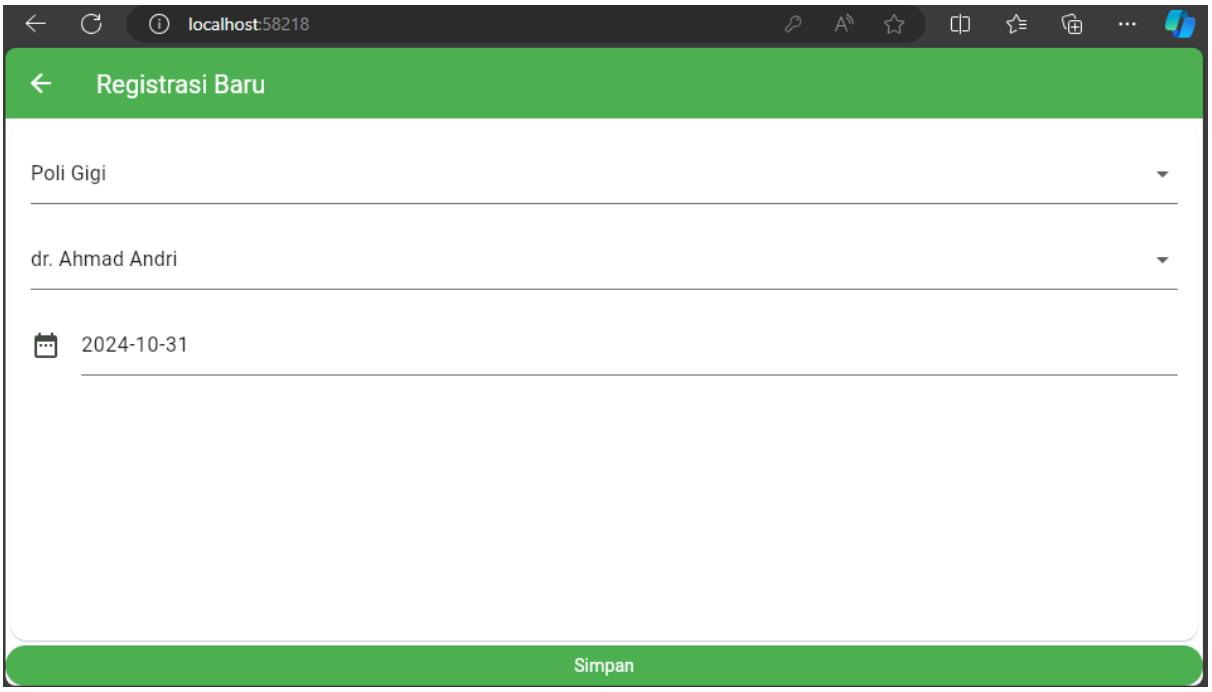
Tampilan pada registrasi poli

Widget RegisPoliList bertanggung jawab untuk menampilkan daftar registrasi poli yang diambil dari state sebelumnya. Penggunaan ListView.builder di sini sangat efisien, karena hanya membuat widget yang diperlukan, menjaga performa aplikasi tetap optimal. GestureDetector ditambahkan, meskipun saat ini tidak ada fungsi yang ditetapkan untuk aksi tap, memberikan peluang untuk memperluas fitur di masa mendatang. Kartu yang digunakan untuk setiap item juga memberikan antarmuka yang bersih dan terorganisir.



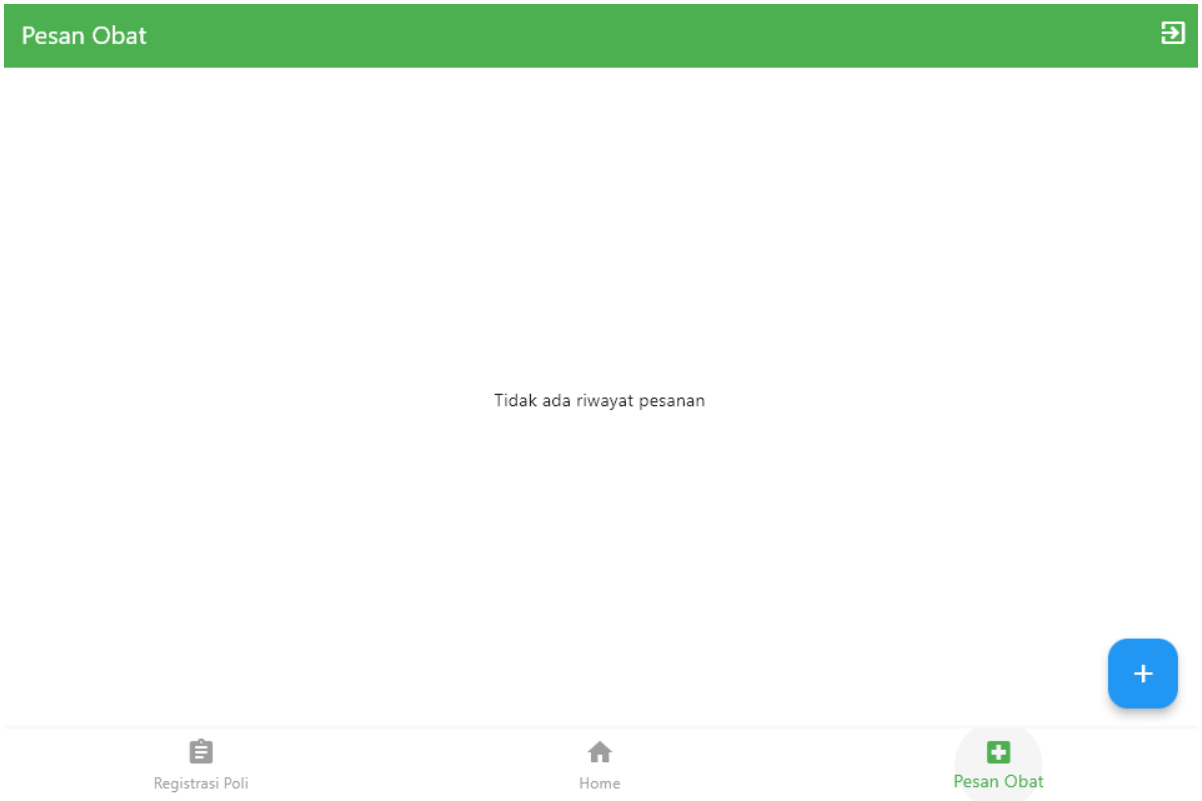
Daftar entri pada registrasi poli

Dalam file create.dart, halaman untuk membuat registrasi baru diimplementasikan. StatefulWidget ini mengelola pemilihan dokter dan poli serta input tanggal. Penggunaan FutureBuilder untuk memuat daftar dokter menunjukkan pendekatan reaktif yang baik, di mana dropdown untuk pemilihan dokter diisi dengan data asinkron. Proses penyimpanan registrasi baru dilakukan dengan memanggil fungsi saveRegisPoli, dan ada penanganan error serta logging untuk memberikan umpan balik.

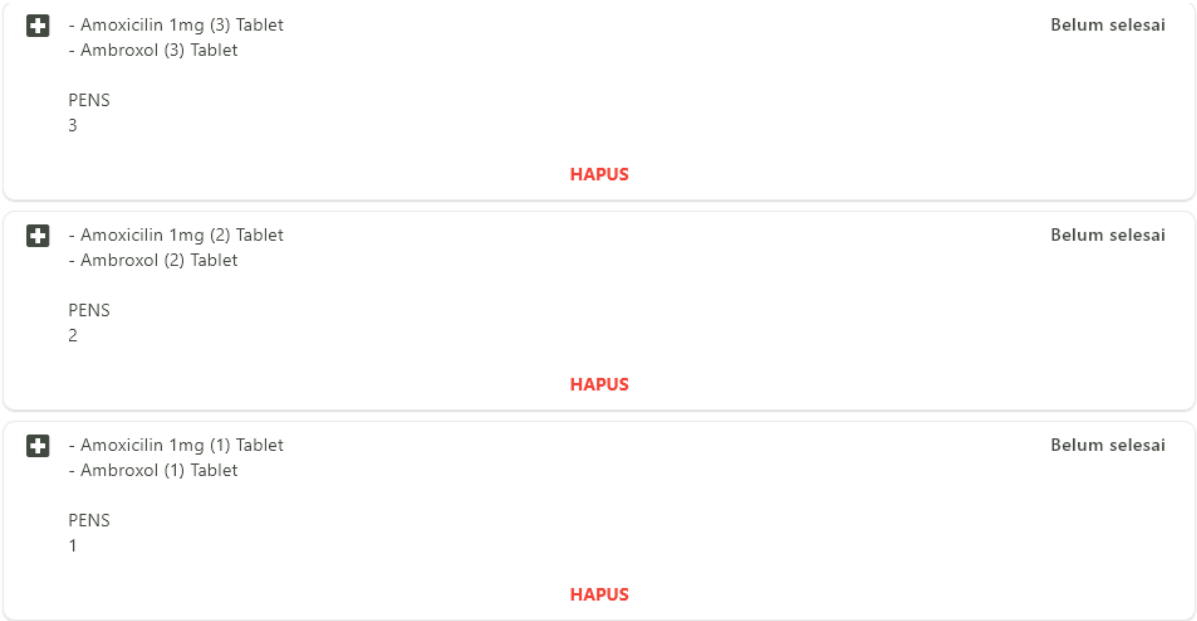


Tampilan registrasi baru

Mirip dengan reg_poli_content.dart, file pesan_obat_content.dart berfungsi untuk mengelola daftar pemesanan obat. FutureBuilder digunakan untuk memuat data pemesanan obat secara asinkron. FloatingActionButton memberikan akses mudah untuk menambah pemesanan baru, sementara pengelolaan hasil yang diterima dari halaman pemesanan baru melalui Navigator memastikan bahwa UI selalu diperbarui dengan informasi terbaru. Penggunaan WidgetsBinding untuk memastikan dialog muncul setelah frame dibangun.

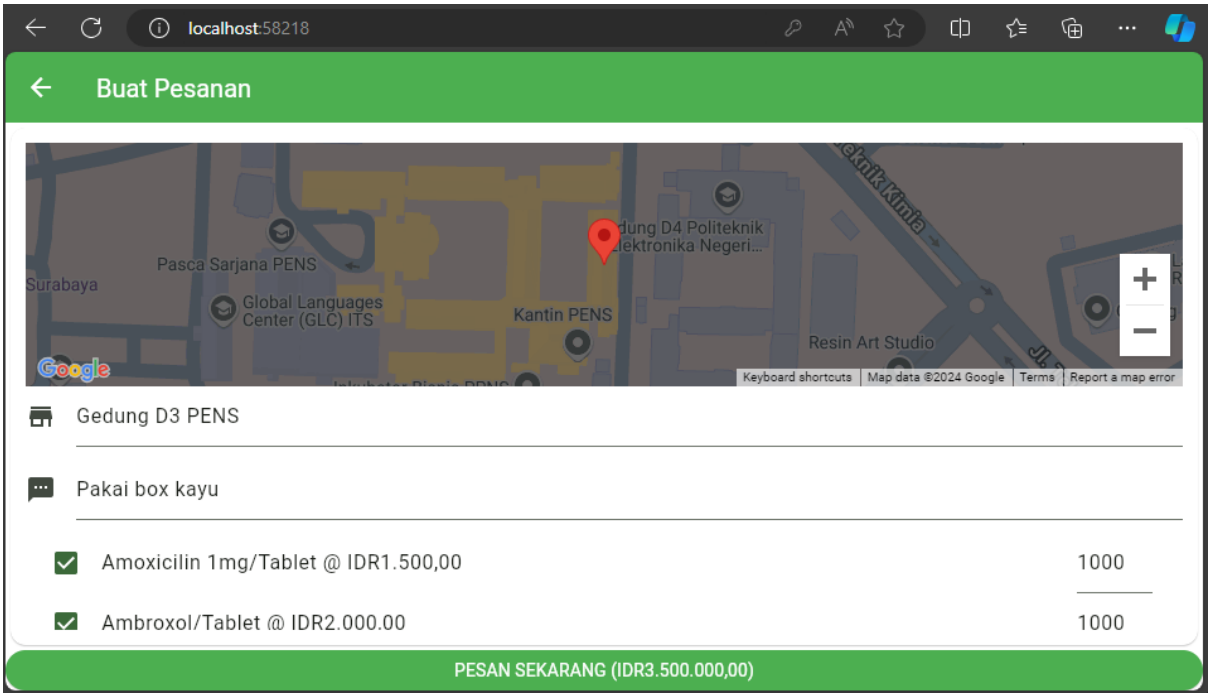


Widget PesanObatList menampilkan daftar pemesanan obat dengan cara yang mirip dengan RegisPoliList. Struktur data disajikan dalam Card dengan ikon yang relevan dan status pemesanan, menciptakan antarmuka yang informatif. Fungsi hapus diimplementasikan menggunakan Visibility, memberikan pengguna kemampuan untuk menghapus pemesanan dengan cara yang bersih dan terorganisir. Penanganan yang baik untuk kondisi data, seperti memeriksa apakah daftar pemesanan kosong, memperkuat pengalaman pengguna yang lebih baik dengan memberikan umpan balik yang sesuai ketika tidak ada data yang ditampilkan.



Daftar entri pada pesan obat

Dalam kode create.dart, terdapat sebuah halaman untuk membuat pesanan obat dengan antarmuka pengguna yang intuitif. Halaman ini menggunakan beberapa elemen dari Flutter, termasuk GoogleMap untuk memilih lokasi dan CheckboxListTile untuk memilih obat yang ingin dipesan. Pengguna dapat mengisi alamat lengkap dan memberikan keterangan tambahan, serta memilih obat dari daftar yang diambil dari API menggunakan fungsi fetchObats(). Ketika pengguna mengklik tombol "PESAN SEKARANG", sistem memvalidasi form untuk memastikan alamat dan obat telah dipilih, lalu mengirimkan data pesanan ke server melalui fungsi pesanObatCreate(). Logger digunakan untuk mencatat aktivitas dan respon dari server, memungkinkan pengembang untuk melacak dan menangani kesalahan yang terjadi selama proses pemesanan. Selain itu, total biaya dikelola secara dinamis berdasarkan jumlah dan harga obat yang dipilih, memberikan transparansi kepada pengguna mengenai total biaya sebelum melakukan pemesanan.



Tampilan ketika memesan obat

Berikut merupakan alur lengkap dari aplikasi ini:

Dimulai dengan proses registrasi pengguna yang memungkinkan calon pasien untuk membuat akun baru. Pengguna mengisi formulir registrasi yang mencakup informasi pribadi seperti nama, email, dan nomor telepon. Setelah registrasi, pengguna dapat memilih untuk mendaftar pada poli tertentu, seperti poli umum atau spesialis, dengan tujuan untuk mendapatkan layanan medis yang sesuai dengan kebutuhan mereka. Setelah mendaftar, pengguna dapat melakukan login ke aplikasi dan mengakses berbagai fitur, termasuk membuat pesanan obat. Dalam proses pemesanan, pengguna mengisi informasi alamat, memberikan keterangan tambahan, dan memilih obat dari daftar yang tersedia. Setiap pilihan obat dilengkapi dengan checkbox untuk memudahkan pengguna memilih sesuai kebutuhan.

Setelah pengguna mengonfirmasi pesanan, informasi tersebut akan dikirim ke server untuk diproses. Di sisi pegawai, aplikasi akan menampilkan pesanan yang masuk, termasuk detail pasien dan obat yang dipesan. Pegawai kemudian memproses pesanan dengan memverifikasi ketersediaan obat dan mengatur pengiriman sesuai alamat yang diberikan. Setelah obat siap dan dikirim, pegawai menandai pesanan sebagai selesai dalam sistem.

E. KESIMPULAN

Kesimpulan pada praktikum ini, aplikasi dapat berjalan dengan baik sebagai solusi mobile client-server yang mengintegrasikan frontend berbasis Flutter dengan backend REST API yang dibangun menggunakan PHP pada platform Android, Web, serta Windows (dekstop). Melalui implementasi model data yang jelas dan terstruktur, aplikasi memungkinkan pengguna untuk melakukan registrasi, login, pendaftaran poli, dan pemesanan obat dengan antarmuka yang intuitif. Fitur-fitur tambahan seperti penyimpanan sesi pengguna (user pasien ataupun pegawai) dan integrasi API Google Maps meningkatkan pengalaman pengguna, sementara penggunaan FutureBuilder memastikan data ditampilkan secara asinkron dan efisien. Menjadikannya aplikasi dengan pengelolaan data yang baik dan alur pengguna yang terorganisir.